



DevOps



Notes By [S.Vamshi](#) 

UNITS	PAGE NO:
1. Introduction To DevOps	2
2. Software development models and DevOps	23
3. Introduction to project management	41
4. Integrating the system	57
5. Testing Tools and automation	73

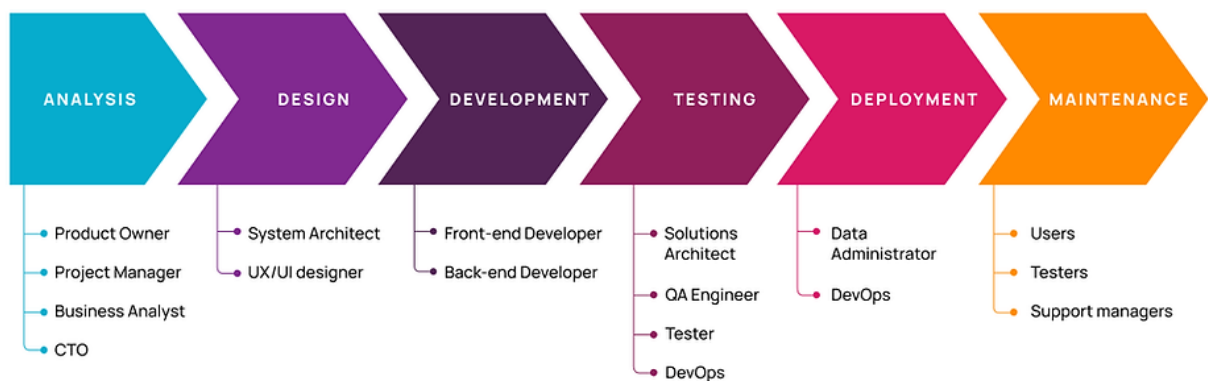
Note: Click on the unit name hyperlinks to navigate to pages!

DevOps UNIT-1

Introduction to Software development

Software development is the process of creating, designing, deploying, and supporting software. It is a systematic and disciplined approach to software creation that aims to create high-quality, reliable, and maintainable software.

6 Phases of the Software Development Life Cycle



The software development process typically involves the following phases:

- **Planning:** This phase involves gathering requirements, defining the scope of the project, and creating a project plan.
- **Requirements analysis:** This phase involves understanding the needs of the users and stakeholders, and documenting the requirements of the software.
- **Design:** This phase involves creating the architecture and design of the software.
- **Implementation:** This phase involves writing the code for the software.
- **Testing:** This phase involves testing the software to ensure that it meets the requirements and is free of errors.
- **Deployment:** This phase involves making the software available to users.
- **Maintenance:** This phase involves fixing errors, adding new features, and updating the software as needed.

There are many different software development methodologies, which are different approaches to the software development process. Some of the most common methodologies include:

- **Waterfall:** This is a traditional methodology that follows a linear approach to development.
- **Agile:** This is a more iterative and incremental methodology that allows for changes to be made to the software throughout the development process.
- **Scrum:** This is a specific type of agile methodology that uses short sprints to develop the software.
- **DevOps:** This is a set of practices that combines software development (Dev) and IT operations (Ops) to shorten the systems development life cycle while delivering high-quality software. DevOps aims at establishing a culture and environment where building, testing, and releasing software can happen rapidly, frequently, and more reliably.

Here is a table that compares DevOps to Waterfall and Agile:

Methodology	Approach	Collaboration	Automation	Benefits
Waterfall	Linear	Low	Low	Predictable, stable
Agile	Iterative, incremental	High	Medium	Flexible, responsive
DevOps	Iterative, incremental	High	High	Fast, reliable, secure

As you can see, DevOps combines the best aspects of Waterfall and Agile. It is a more collaborative and automated approach to software development that can help organizations deliver software more quickly, reliably, and securely.

Here are some of the benefits of DevOps:

- **Increased speed:** DevOps can help organizations deliver software more quickly by automating tasks and breaking down silos between development and operations teams.
- **Improved reliability:** DevOps can help organizations improve the reliability of their software by implementing continuous integration and continuous delivery (CI/CD) pipelines.

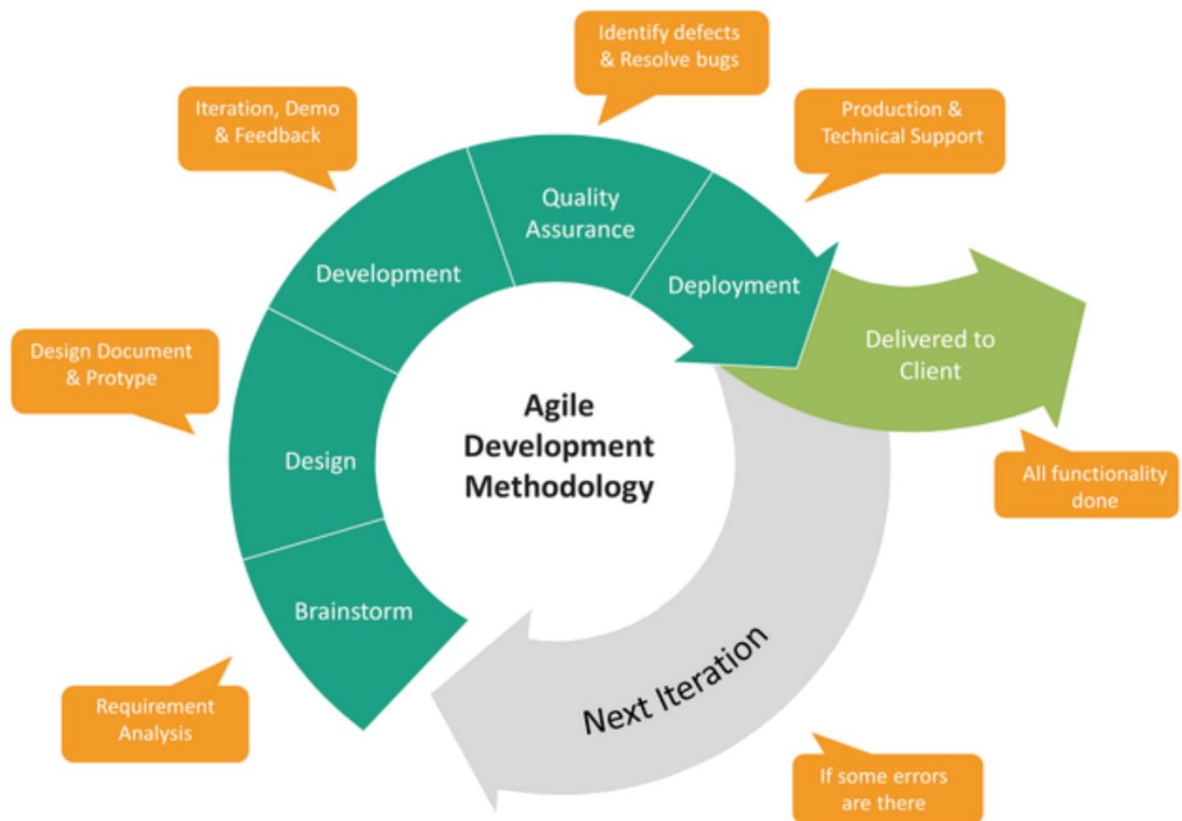
- **Increased security:** DevOps can help organizations improve the security of their software by implementing automated security testing and continuous monitoring.
- **Reduced costs:** DevOps can help organizations reduce the costs of software development by automating tasks and eliminating waste.

Agile development model

Agile development is a software development methodology that emphasizes flexibility, collaboration, and iterative development. It is a response to the limitations of traditional waterfall models, which follow a sequential and rigid approach. Agile methodologies, such as Scrum, Kanban, and Extreme Programming (XP), prioritize adaptability, customer satisfaction, and continuous improvement.

Here are some key characteristics and principles of Agile development:

1. **Iterative and Incremental:** Agile projects are divided into small iterations or time-boxed sprints, typically ranging from one to four weeks. Each iteration results in a potentially shippable product increment, allowing for early feedback and continuous improvement.
2. **Customer Collaboration:** Agile development encourages close collaboration between the development team and the customer or product owner. Frequent communication and feedback loops ensure that the software meets the customer's expectations and evolving needs.
3. **Adaptive Planning:** Agile projects embrace change and prioritize responding to new requirements or insights. Rather than detailed upfront planning, Agile teams create flexible plans that accommodate changing priorities and emerging information throughout the project.
4. **Self-Organizing Teams:** Agile development promotes self-organizing teams where members collaborate, share responsibilities, and make collective decisions. This fosters a sense of ownership and empowerment, leading to increased motivation and productivity.
5. **Continuous Integration and Delivery:** Agile teams emphasize continuous integration, where developers regularly merge their work to detect and resolve integration issues early. Continuous delivery allows for frequent and reliable releases, ensuring that valuable software features are available to end-users as soon as possible.
6. **Empirical Process Control:** Agile development relies on empirical feedback loops, such as regular retrospectives, to continuously assess and improve the team's performance, processes, and product quality.

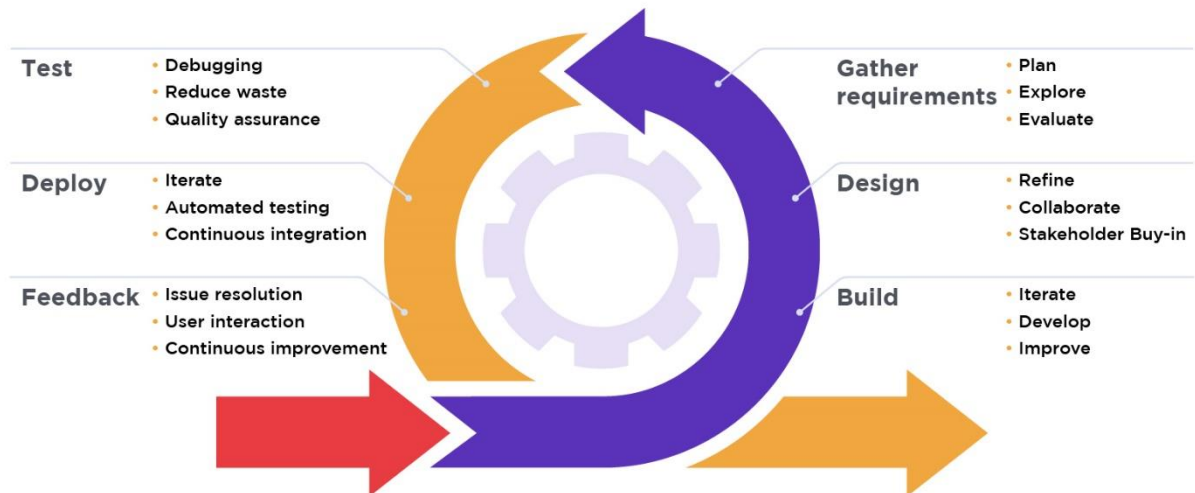


Agile methodologies offer several benefits:

- **Flexibility and Adaptability:** Agile development allows for changing requirements and priorities, ensuring that the software aligns with the evolving needs of the customer or market.
- **Early and Continuous Feedback:** Regular iterations and customer collaboration enable quick feedback loops, helping to identify issues early and incorporate improvements throughout the development process.
- **Increased Transparency:** Agile methodologies promote transparency through frequent communication, visible progress, and open collaboration, ensuring that stakeholders are informed and engaged throughout the project.
- **Reduced Risks:** The iterative nature of Agile development reduces risks by addressing potential issues early, enabling course correction, and minimizing the impact of changes or uncertainties.

To implement Agile successfully, teams often employ project management frameworks like Scrum, which provides specific roles, artifacts (e.g., product backlog, sprint backlog), and ceremonies (e.g., daily stand-ups, sprint planning, sprint review) to structure the development process.

Stages of Agile Modelling



In the Agile model, software development is divided into several stages, each contributing to the iterative and incremental delivery of the software. The specific names and terminology used may vary depending on the Agile methodology being followed (e.g., Scrum, Kanban), but the underlying principles remain consistent. Here are the common stages in Agile development:

1. **Product Backlog:** The product backlog is the initial stage where the project's requirements, features, and user stories are collected and prioritized. The product owner, in collaboration with stakeholders, creates and maintains the backlog, which serves as a dynamic list of items to be addressed during the development process.
2. **Sprint Planning:** In this stage, the development team selects a set of items from the product backlog to work on during the upcoming sprint. The team and the product owner collaborate to understand the requirements, break them down into smaller tasks, estimate the effort involved, and determine the sprint goal.
3. **Sprint:** A sprint is a time-boxed iteration typically lasting from one to four weeks. During the sprint, the development team focuses on delivering the selected backlog items. Daily stand-up meetings are conducted to synchronize the team's activities, discuss progress, and identify any obstacles or impediments.
4. **Development and Testing:** This stage involves the actual development of the software features and their associated testing. Developers write code, following coding standards and best practices, and implement the desired functionality. Automated tests and unit tests are written to verify the correctness of the code and catch potential defects early.
5. **Sprint Review:** At the end of each sprint, a sprint review meeting is held to showcase the completed work to the stakeholders. The development team demonstrates the

implemented features, gathers feedback, and collaborates with the product owner to review and adjust the product backlog based on the stakeholders' input.

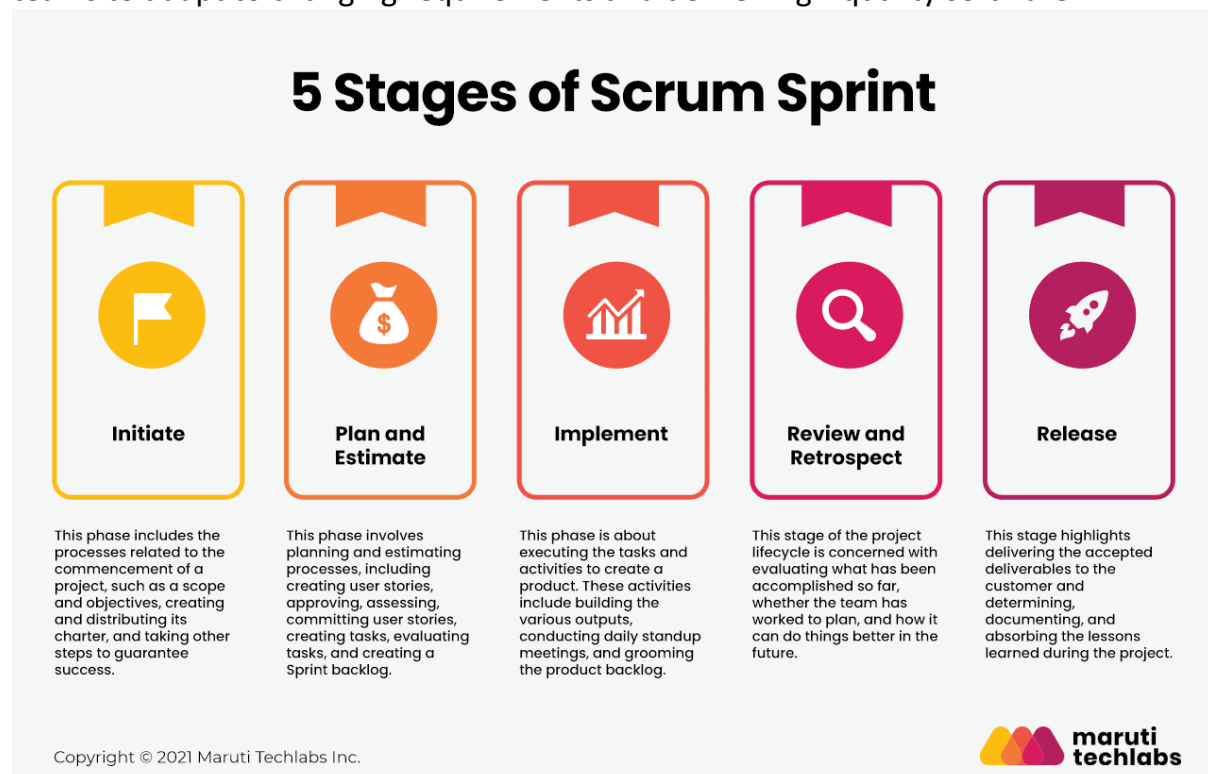
6. **Sprint Retrospective:** The sprint retrospective is a team reflection meeting that takes place after the sprint review. The team discusses what went well, what didn't go well, and identifies areas for improvement. Retrospectives help the team continuously enhance their processes, collaboration, and productivity for future sprints.

7. **Repeat:** After the sprint retrospective, the next sprint planning begins, and the cycle repeats. The team selects a new set of backlog items, works on their development and testing, conducts a sprint review, and holds a retrospective. This iterative process continues until the desired software is fully developed or the project's objectives are achieved.

Throughout the Agile model, continuous communication, collaboration, and feedback play a vital role. The iterative nature of Agile allows for regular inspection and adaptation, ensuring that the software aligns with the changing requirements and the customer's needs. The stages in Agile development are designed to facilitate flexibility, transparency, and early delivery of valuable software increments.

Scrum in Agile

Scrum is an Agile framework for managing and organizing software development projects. It provides a structured approach to collaboration, continuous improvement, and iterative delivery. Scrum emphasizes flexibility, self-organization, and rapid feedback loops, allowing teams to adapt to changing requirements and deliver high-quality software.



Key components of Scrum include:

1. **Roles:**

- **Product Owner:** Represents the interests of stakeholders, defines and prioritizes the product backlog, and ensures that the team is delivering value to the customer.
- **Scrum Master:** Facilitates the Scrum process, removes obstacles that hinder the team's progress, and ensures adherence to Scrum principles and practices.
- **Development Team:** A self-organizing, cross-functional group responsible for delivering the product increment. They collaborate to plan, develop, test, and deliver the work.

2. **Artifacts:**

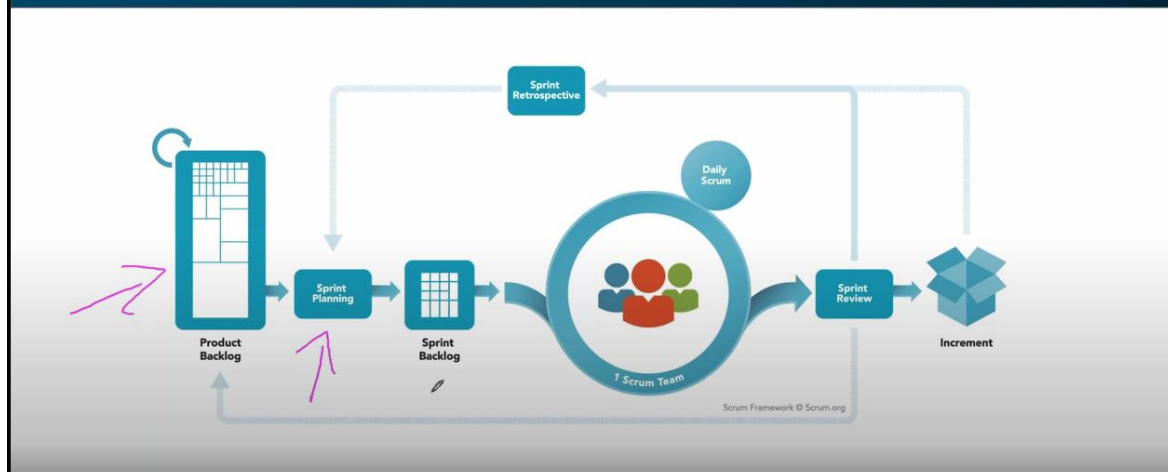
- **Product Backlog:** A prioritized list of features, user stories, and tasks that represent the requirements for the product. It is maintained and managed by the product owner and evolves throughout the project.
- **Sprint Backlog:** A subset of the product backlog items selected for a specific sprint. It contains the work the development team commits to delivering during the sprint.
- **Increment:** The sum of all the completed product backlog items at the end of a sprint, potentially shippable and meeting the definition of done.

3. **Ceremonies:**

- **Sprint Planning:** The team collaboratively plans the work to be done in the upcoming sprint. They review the product backlog, select items for the sprint backlog, and define a sprint goal.
- **Daily Scrum:** A brief daily meeting where the team synchronizes their activities, shares progress, discusses impediments, and plans their work for the day.
- **Sprint Review:** At the end of each sprint, the team presents the completed work to stakeholders, obtains feedback, and discusses future priorities. Adjustments to the product backlog are made based on the feedback.
- **Sprint Retrospective:** The team reflects on the just-concluded sprint and identifies what went well, what could be improved, and actionable items for the next sprint. It focuses on process improvement and adaptation.

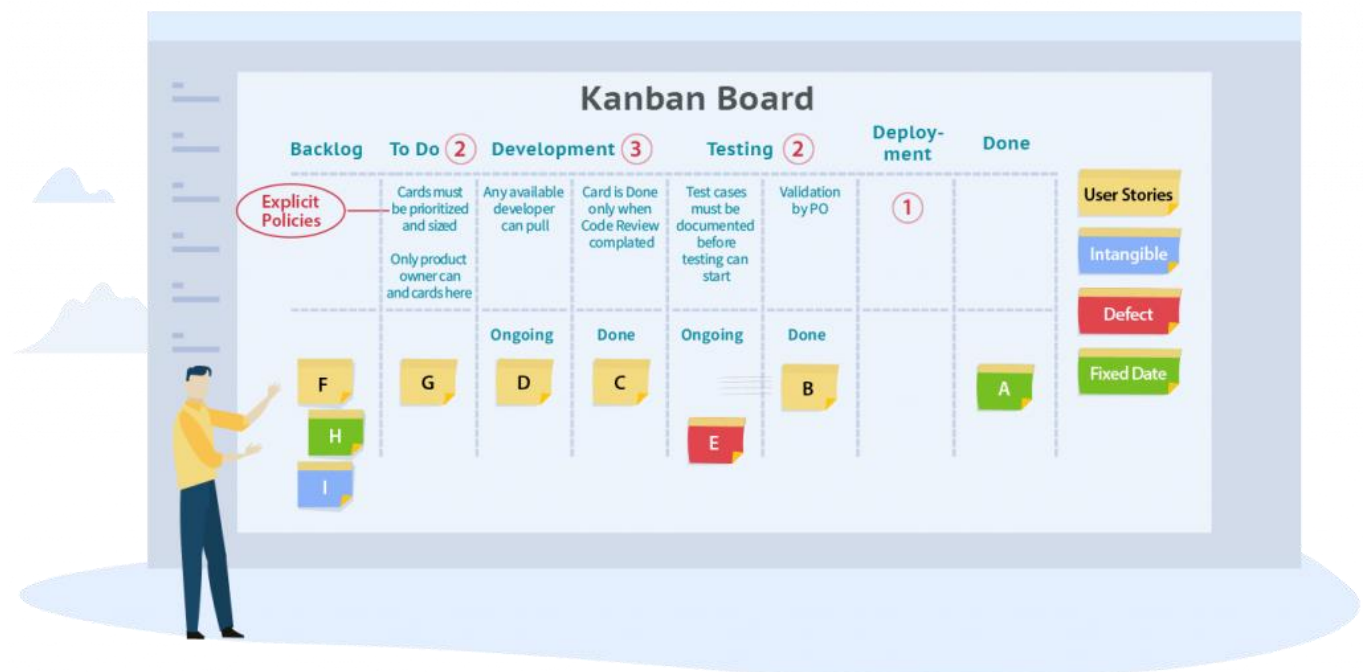
Scrum promotes a time-boxed approach with fixed-duration iterations called sprints, typically lasting two to four weeks. The team aims to deliver a potentially releasable increment of the product at the end of each sprint. Sprints enable quick feedback, adaptation, and continuous improvement throughout the development process.

Scrum's iterative and incremental nature allows for greater visibility, transparency, and collaboration. It fosters self-organizing teams and encourages regular inspection and adaptation to maximize productivity, quality, and customer satisfaction.



kanban in agile

Kanban is an Agile framework that focuses on visualizing and optimizing the flow of work. It originated from the manufacturing industry and was later adapted for software development. Kanban provides a flexible approach to managing tasks and improving efficiency by emphasizing a continuous and smooth workflow.



Key concepts and principles of Kanban include:

1. **Visualization:** Kanban utilizes visual boards, often represented as a series of columns, to represent the workflow. Each column represents a stage of work, such as "To

Do," "In Progress," and "Done." Tasks or user stories are represented as cards or sticky notes and move across the board as they progress through the workflow.

2. ****Work in Progress (WIP) Limit:**** Kanban sets a maximum limit for the number of tasks that can be in progress at each stage of the workflow. This limit helps prevent overloading the team, reduces multitasking, and promotes focus on completing tasks before starting new ones.
3. ****Pull System:**** Kanban follows a pull-based approach, where tasks are pulled into the workflow only when there is available capacity. This ensures that work is initiated based on actual capacity and that team members don't become overwhelmed.
4. ****Continuous Improvement:**** Kanban encourages the team to reflect on their processes and continuously seek ways to improve efficiency, quality, and flow. Through regular analysis and adaptation, the team identifies bottlenecks, inefficiencies, and opportunities for optimization.
5. ****Metrics and Data-Driven Decision Making:**** Kanban emphasizes the use of data and metrics to track and monitor the flow of work. Metrics such as lead time, cycle time, and throughput are measured and analyzed to identify areas for improvement and make data-driven decisions.

Kanban does not prescribe specific ceremonies or roles like Scrum does. Instead, it provides a framework for visualizing work and optimizing workflow. It can be implemented within teams using other Agile methodologies or even in non-Agile environments.

The benefits of Kanban in Agile include:

- ****Transparency:**** Kanban provides clear visibility into the work in progress, allowing team members and stakeholders to understand the status of tasks and identify potential bottlenecks.
- ****Flexibility:**** Kanban allows for the dynamic reprioritization of work and the ability to handle urgent tasks or changes without disrupting the overall flow.
- ****Efficiency:**** By setting WIP limits and optimizing workflow, Kanban helps teams focus on completing tasks and reduces the time spent on multitasking and context switching.
- ****Continuous Improvement:**** Kanban promotes a culture of continuous improvement by encouraging regular reflection and adaptation based on data-driven insights.

Kanban is particularly suitable for teams with a continuous stream of incoming work, where the focus is on maintaining a steady flow rather than time-boxed iterations. It can be used in various domains and industries to manage and optimize workflows effectively.

Continuous delivery ensures that software changes are regularly deployed to production environments in a reliable and automated manner.

4. **Infrastructure as Code (IaC):** Infrastructure as Code treats infrastructure provisioning, configuration, and management as software artifacts. It allows teams to define and manage infrastructure resources using code, enabling version control, reproducibility, and consistency across environments.

5. **Monitoring and Feedback:** DevOps emphasizes continuous monitoring of applications, infrastructure, and user feedback. Monitoring helps identify issues, performance bottlenecks, and potential improvements. Feedback loops are established to gather user input and insights, enabling teams to iterate and improve their software based on real-world usage.

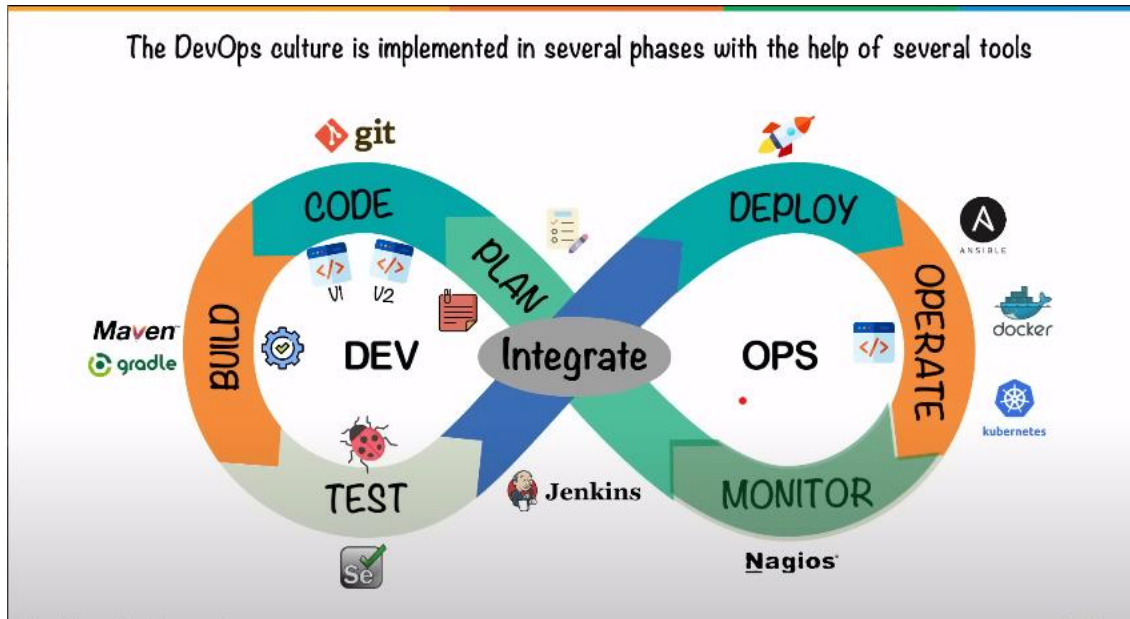
6. **DevSecOps:** DevSecOps integrates security practices into the DevOps process, ensuring that security is prioritized and integrated throughout the software development lifecycle. Security considerations are addressed from the early stages of development to deployment and ongoing operations.

The benefits of DevOps include:

- **Faster Time to Market:** DevOps enables faster software development and deployment cycles, reducing time to market for new features and enhancements.
- **Improved Collaboration:** DevOps breaks down silos between teams, fostering collaboration and communication, leading to better alignment, shared understanding, and faster problem resolution.
- **Increased Efficiency and Quality:** Automation and streamlined processes in DevOps improve efficiency, reduce manual errors, and enhance the overall quality of software releases.
- **Greater Stability and Reliability:** DevOps practices such as infrastructure as code and continuous delivery help ensure more stable and reliable software deployments.
- **Continuous Improvement:** DevOps promotes a culture of continuous improvement and learning, enabling teams to iterate and enhance their processes, products, and customer experiences.

DevOps is not a specific tool or technology but rather a philosophy and set of practices that organizations adopt to create a more collaborative and efficient software development and deployment ecosystem.

Lifecycle of DevOps



The DevOps lifecycle consists of several phases that encompass the software development, deployment, and operations processes. While the specific names and practices may vary depending on the organization, here are the common phases in the DevOps lifecycle:

1. **Plan:** In this phase, teams define the overall objectives, requirements, and scope of the software project. It involves collaborating with stakeholders, understanding user needs, and establishing a roadmap for development and deployment.
2. **Develop:** The development phase involves writing code, building software components, and integrating different modules or services. Development teams follow coding best practices, version control, and collaborate to create the desired features and functionalities.
3. **Build:** In this phase, the software is compiled, built, and packaged into a deployable format. It may involve activities such as compiling source code, running tests, and creating artifacts that can be deployed to various environments.
4. **Test:** The testing phase focuses on verifying the quality and functionality of the software. Different types of testing, including unit testing, integration testing, and performance testing, are performed to identify and address defects and ensure the software meets the required standards.
5. **Deploy:** The deployment phase involves releasing the software to production or target environments. It includes activities like configuring infrastructure, provisioning resources, and deploying the built artifacts. DevOps practices such as infrastructure as code and automation help ensure consistent and reliable deployments.
6. **Operate:** Once the software is deployed, it enters the operation phase. This phase involves monitoring the software's performance, availability, and user experience. It includes activities like log analysis, performance monitoring, error tracking, and incident

management. Continuous monitoring and feedback help identify and address issues promptly.

7. ****Monitor:**** The monitoring phase involves collecting and analyzing data on various aspects of the software, infrastructure, and user interactions. Monitoring tools and techniques are employed to track metrics, identify performance bottlenecks, and gain insights for continuous improvement.

8. ****Iterate/Improve:**** DevOps is an iterative and continuous improvement process. Teams analyze the data, user feedback, and operational insights gathered from monitoring to identify areas for improvement. They apply this knowledge to refine the software, infrastructure, and processes, enhancing performance, stability, and user experience.

Throughout the DevOps lifecycle, automation, collaboration, and continuous integration and delivery play essential roles. DevOps teams utilize tools and technologies for automation, version control, continuous integration, deployment pipelines, and monitoring to streamline and optimize the software development and deployment processes.

It's important to note that the DevOps lifecycle is not a linear sequence of steps, but rather an interconnected and iterative process. It emphasizes frequent collaboration, feedback, and continuous improvement to ensure the delivery of high-quality software that meets customer needs and business objectives.

DevOps Process:

DevOps Process

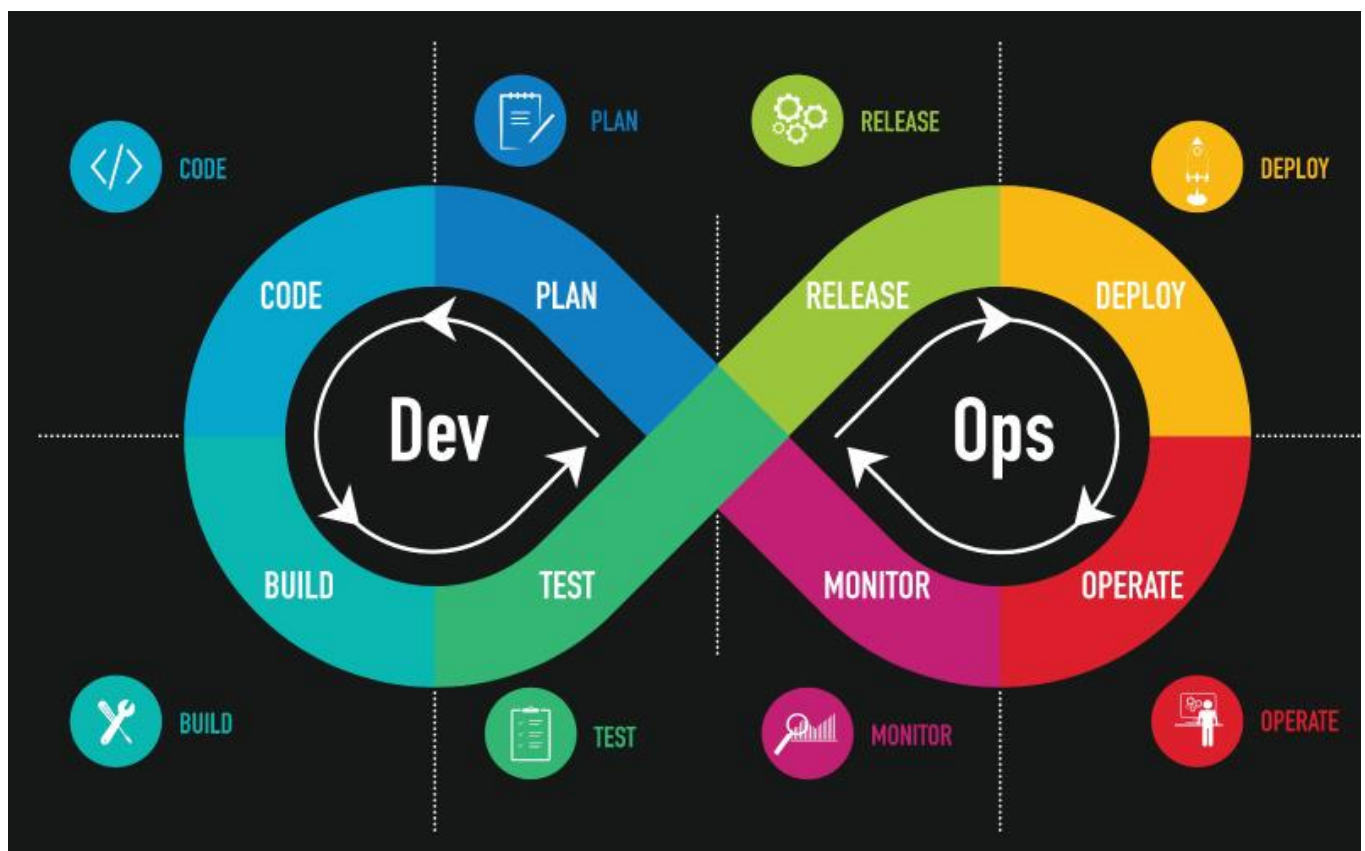
The DevOps process refers to the practices and methodologies employed to enable collaboration and integration between development and operations teams, with the goal of delivering software quickly, reliably, and efficiently. Here are some key aspects of the DevOps process:

1. ****Collaboration and Communication:**** DevOps emphasizes cross-functional collaboration, effective communication, and shared responsibility among teams. Developers, testers, operations personnel, and other stakeholders work together to align objectives, share knowledge, and resolve issues.
2. ****Automation:**** Automation is a fundamental aspect of DevOps. It involves using tools and technologies to automate repetitive tasks, such as code compilation, testing, deployment, and infrastructure provisioning. Automation reduces manual errors, accelerates processes, and ensures consistency.
3. ****Continuous Integration (CI):**** CI is a practice where developers regularly integrate their code changes into a shared repository. Automated builds and tests are triggered to validate the code, ensuring that it is compatible with the existing codebase. CI detects issues early, facilitates collaboration, and promotes a more stable codebase.
4. ****Continuous Delivery (CD):**** CD is the practice of frequently delivering software changes to production or other target environments. It involves automating the deployment process, enabling teams to release new features and enhancements more

frequently. CD ensures that software changes are ready for deployment and can be released quickly and reliably.

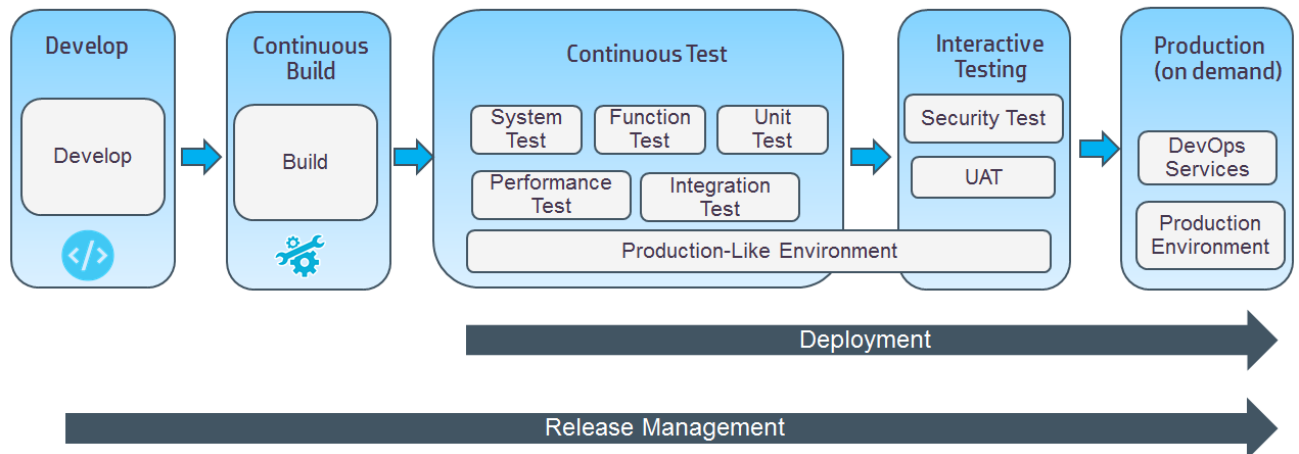
5. **Infrastructure as Code (IaC):** IaC treats infrastructure provisioning and management as code. Infrastructure resources, such as servers, networks, and databases, are defined and managed through code and version control systems. IaC enables consistent, repeatable, and automated infrastructure deployment and configuration.

6. **Monitoring and Feedback:** DevOps emphasizes continuous monitoring of applications, infrastructure, and user feedback. Monitoring tools track various metrics, such as performance, availability, and user experience, to identify issues and enable prompt remediation. User feedback is collected to understand user needs, drive improvements, and prioritize feature development.





DevOps Continuous Delivery Process



Continuous Delivery

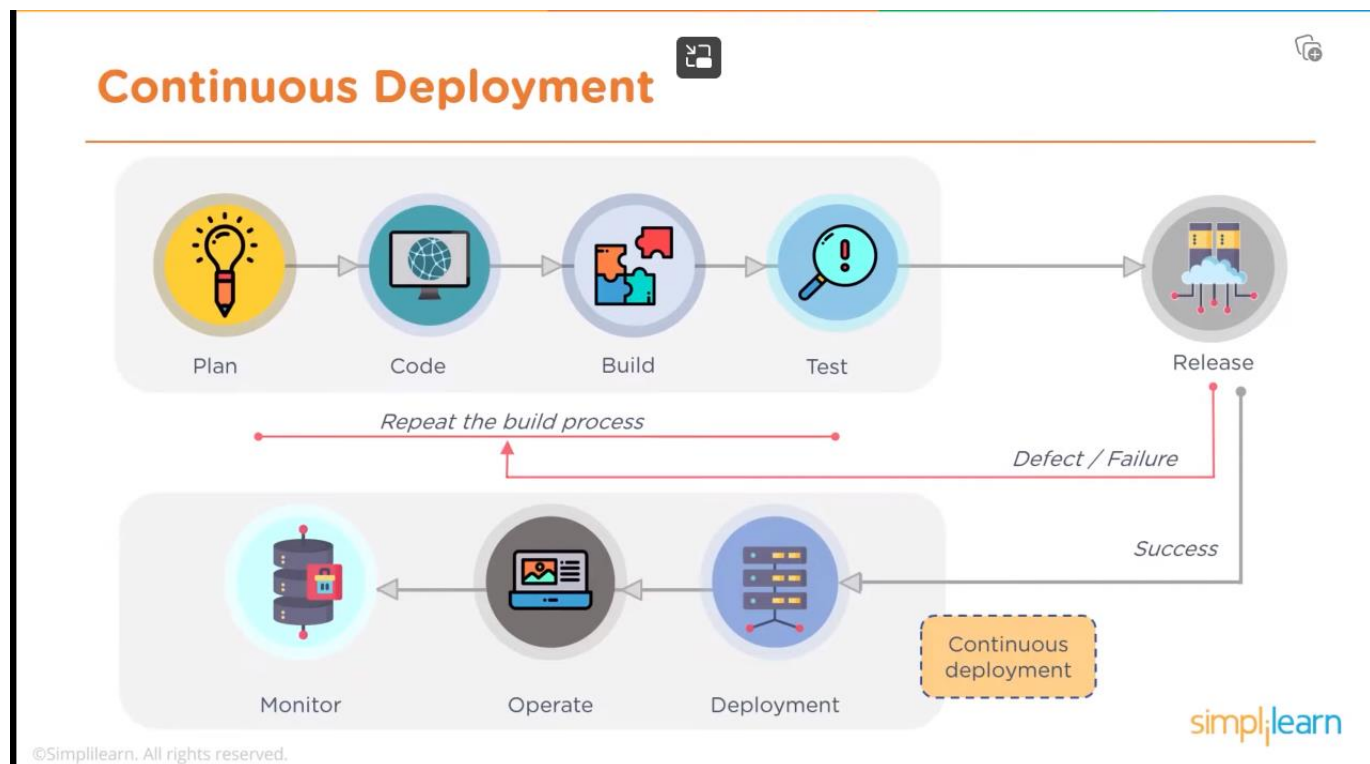
Continuous Delivery is a software development practice that extends continuous integration to enable the frequent and reliable delivery of software changes to production or other target environments. It focuses on minimizing the time and effort required to make software changes ready for release. Key aspects of Continuous Delivery include:

1. **Build Automation:** Continuous Delivery automates the build process, ensuring that software changes are compiled, tested, and packaged consistently and reproducibly. This automation removes manual steps and reduces the risk of errors.
2. **Deployment Automation:** Continuous Delivery automates the deployment process, enabling the rapid and repeatable deployment of software changes to different environments. Automation reduces the chances of deployment errors and ensures consistency across environments.
3. **Configuration Management:** Continuous Delivery emphasizes the use of configuration management tools and techniques to manage application configurations across different environments. Configuration files and settings are versioned, making it easier to manage and deploy changes consistently.
4. **Continuous Testing:** Continuous Delivery requires comprehensive testing practices. Automated testing is performed throughout the development and deployment pipeline to ensure that software changes are thoroughly validated for functionality, compatibility, and performance.
5. **Release Orchestration:** Continuous Delivery involves orchestrating the release process, managing the flow of software changes from development to production.

This includes activities such as release planning, change management, and coordination between development and operations teams.

6. ****Incremental and Safe Releases:**** Continuous Delivery promotes releasing software changes incrementally and safely, rather than in large batches. It encourages techniques like feature toggles, canary releases, and A/B testing to gradually introduce changes and validate them in a controlled manner.

Continuous Delivery allows organizations to respond quickly to market demands, reduce the risk of software releases, and obtain fast feedback from users. It promotes a culture of frequent, reliable, and automated software delivery, enabling organizations to deliver value to customers efficiently



For more information regarding DevOps click on the below link:

[Introduction To DevOps](#) | [Devops Tutorial For Beginners](#) | [DevOps Training For Beginners](#) | [Simplilearn](#)

ITIL (Information Technology Infrastructure Library) is a set of best practices for IT service management (ITSM). It provides guidance on how to deliver high-quality IT services that meet the needs of the business. ITIL can be used to improve the efficiency and effectiveness of software development by providing a structured approach to the following:

- Requirements gathering and analysis: ITIL provides a framework for gathering and analyzing user requirements, ensuring that the software being developed meets the needs of the business.
- Design: ITIL provides guidance on the design of software, ensuring that it is scalable, reliable, and secure.
- Development: ITIL provides guidance on the development of software, ensuring that it is developed in a consistent and controlled manner.
- Testing: ITIL provides guidance on the testing of software, ensuring that it is free of defects and meets the requirements of the business.
- Deployment: ITIL provides guidance on the deployment of software, ensuring that it is deployed in a way that minimizes disruption to the business.
- Operation and maintenance: ITIL provides guidance on the operation and maintenance of software, ensuring that it continues to meet the needs of the business.

ITIL can be used in conjunction with other software development methodologies, such as Agile and Waterfall. By integrating ITIL into the software development process, organizations can improve the quality, reliability, and efficiency of their software.

Here are some of the benefits of using ITIL in software development:

- Improved quality: ITIL can help to improve the quality of software by providing a structured approach to requirements gathering, design, development, testing, deployment, and operation.
- Reduced costs: ITIL can help to reduce the costs of software development by improving efficiency and effectiveness.
- Increased customer satisfaction: ITIL can help to increase customer satisfaction by delivering software that meets their needs.

If you are looking for a way to improve the quality, reliability, and efficiency of your software development process, then ITIL is a good option to consider.

Release Management

Release management in DevOps refers to the process of planning, coordinating, and deploying software releases to production or other target environments. It focuses on ensuring that software changes are released in a controlled, efficient, and reliable manner. Release management in DevOps aims to streamline the release process, minimize disruptions, and provide transparency and traceability throughout the lifecycle.

Here are key aspects of release management in DevOps:

1. **Release Planning:** Release management starts with planning, where the scope, content, and timing of the release are determined. The release plan considers factors such as business priorities, customer needs, available resources, and potential risks. It includes defining release objectives, identifying features or enhancements, and setting a timeline.
2. **Release Coordination and Communication:** Release management involves coordinating activities between development, testing, operations, and other stakeholders involved in the release process. It ensures that everyone is aligned, aware of their responsibilities, and communicates effectively throughout the release cycle. Collaboration tools and practices facilitate coordination and enable real-time communication.
3. **Version Control and Configuration Management:** Effective release management relies on version control and configuration management practices. It ensures that the correct versions of software components, configurations, and dependencies are tracked and managed. Version control systems maintain a history of changes, enabling easy rollback and traceability.
4. **Release Packaging and Deployment:** The release management process includes packaging software changes into deployable units, such as software builds or container images. Packaging ensures that all necessary files, dependencies, and configurations are included. Automated deployment practices, such as infrastructure as code and deployment pipelines, help streamline and standardize the deployment process.
5. **Testing and Validation:** Release management incorporates testing and validation activities to ensure that software changes meet quality standards and are compatible with the target environment. This includes various testing practices, such as unit testing, integration testing, regression testing, and performance testing. Automated testing helps identify and address issues early in the release process.

6. **Release Automation:** Automation plays a crucial role in release management. It involves automating repetitive and error-prone tasks, such as building, packaging, and deploying software changes. Automation ensures consistency, reduces manual effort, and minimizes the risk of human errors during the release process. Continuous integration and continuous delivery (CI/CD) pipelines are commonly used for release automation.

7. **Release Monitoring and Feedback:** Release management includes monitoring and collecting feedback on the released software. Monitoring tools track the performance, availability, and usage of the software in production. User feedback and support channels provide insights into user experiences and identify areas for improvement. Monitoring and feedback help address issues, prioritize future enhancements, and drive continuous improvement.

8. **Rollback and Rollforward:** Release management should have mechanisms in place to handle rollback and rollforward scenarios. In case of issues or failures during the release, the ability to roll back to the previous version or roll forward to a newer version is crucial. This requires proper planning, version control, and automated deployment processes.

Release management in DevOps aims to balance the need for frequent releases with stability, reliability, and risk mitigation. It emphasizes automation, collaboration, and transparency to ensure successful and controlled software deployments. By adopting effective release management practices, organizations can deliver value to customers faster, reduce downtime, and enhance overall software delivery capabilities.

Delivery pipeline

In DevOps, a delivery pipeline, also known as a deployment pipeline or a CI/CD pipeline (Continuous Integration/Continuous Delivery pipeline), is an automated process that enables the continuous integration, testing, and deployment of software changes from development to production environments. It serves as a structured and repeatable framework for efficiently delivering software updates and new features.

The delivery pipeline consists of several stages or steps, each of which performs specific actions to ensure that the software changes are validated, integrated, and ready for production release. Here are the common stages in a delivery pipeline:

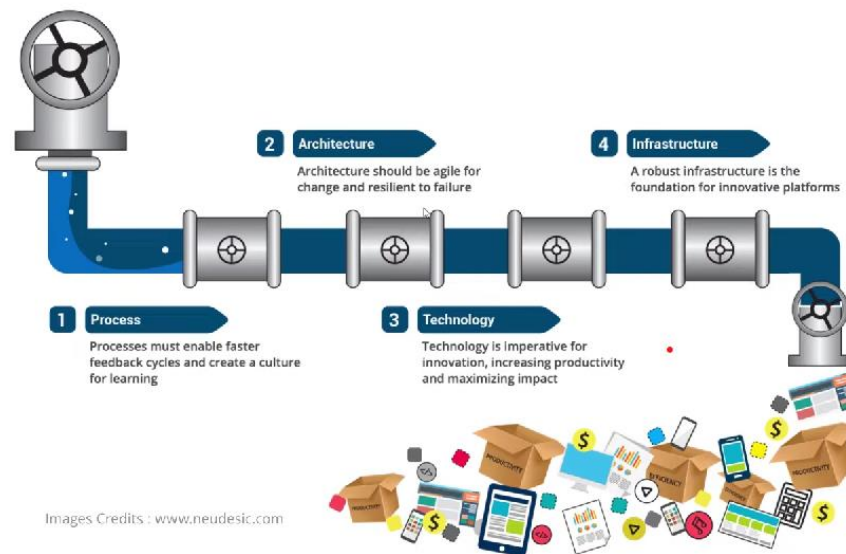
1. **Source Code Management:** The pipeline starts with the source code management stage, where the software code is stored and version controlled using tools like Git. Developers commit their changes to the code repository, triggering the pipeline's execution.

2. ****Build:**** In the build stage, the code is compiled, dependencies are resolved, and the software is built into executable artifacts. Build tools like Maven, Gradle, or npm are commonly used to automate this process. The resulting artifacts are generated and ready for further testing.
3. ****Automated Testing:**** The built artifacts go through various automated testing stages. These can include unit testing, which verifies the functionality of individual components, and integration testing, which ensures that the different components work together correctly. Other types of tests, such as functional, performance, and security testing, may also be performed in this stage.
4. ****Code Quality Analysis:**** Code quality analysis tools are often integrated into the pipeline to assess the code's quality and adherence to coding standards. Static code analysis tools, such as SonarQube or ESLint, are used to identify issues like code smells, security vulnerabilities, or performance bottlenecks.
5. ****Artifact Repository:**** The built artifacts, along with any other required files and configurations, are stored in an artifact repository. This repository serves as a centralized location for storing and managing deployable artifacts. It ensures that the software releases are traceable, reproducible, and easily accessible.
6. ****Deployment:**** The deployment stage involves taking the artifacts generated from the build and testing stages and deploying them to target environments, such as development, staging, or production environments. Infrastructure as Code (IaC) tools, like Terraform or Ansible, may be used to automate infrastructure provisioning and configuration.
7. ****Release and Promotion:**** Once the software passes the automated testing and deployment stages, it can be released to a specific environment. This stage may involve coordinating with operations teams, updating production configurations, or performing database migrations. The software release can be promoted to subsequent environments, such as staging or production, using predefined rules and approvals.
8. ****Monitoring and Feedback:**** Continuous monitoring of the deployed software in the production environment provides insights into its performance, availability, and user experience. Feedback from users and stakeholders is collected to identify areas for improvement and drive future iterations. This feedback can be used to inform the next development cycle and trigger further changes in the pipeline.

The delivery pipeline in DevOps aims to automate and streamline the software delivery process, enabling frequent releases, faster time-to-market, and improved software quality. It reduces manual effort, minimizes errors, and ensures consistent and reliable deployments. The pipeline can be customized and tailored to meet the

specific requirements and practices of an organization, and it plays a vital role in achieving continuous integration and continuous delivery objectives.

Software Development Life-Cycle



©Simplilearn. All rights reserved.

simplilearn

Delivery Pipeline

In DevOps, a bottleneck is a point in the software development process where the flow of work is impeded, causing delays and inefficiencies¹. Bottlenecks can be identified by seeing tickets pile up in a column or by seeing that tickets get pulled out of a column faster than that new tickets come in¹. To achieve incremental software development and continuous feedback, it is important to eliminate the tasks that create waste or bottlenecks¹.

Here are some common bottlenecks in DevOps:

- 1. **Testing****: Testing is a major source of bottlenecks for most DevOps value streams. It is important to optimize testing processes to reduce delays and inefficiencies.
- 2. **Architecture****: The architecture of an application can also be a bottleneck in the DevOps process. It is important to ensure that the architecture is scalable and can handle the demands of the application⁴.
- 3. **Manual intervention****: Manual intervention is not always advisable for all IT processes. It can lead to delays and inefficiencies in the DevOps process⁶.
- 4. **Technical debt****: Technical debt refers to the cost of maintaining outdated legacy systems or using the wrong tools during bug fixes. It can lead to delays and inefficiencies in the DevOps process⁵.

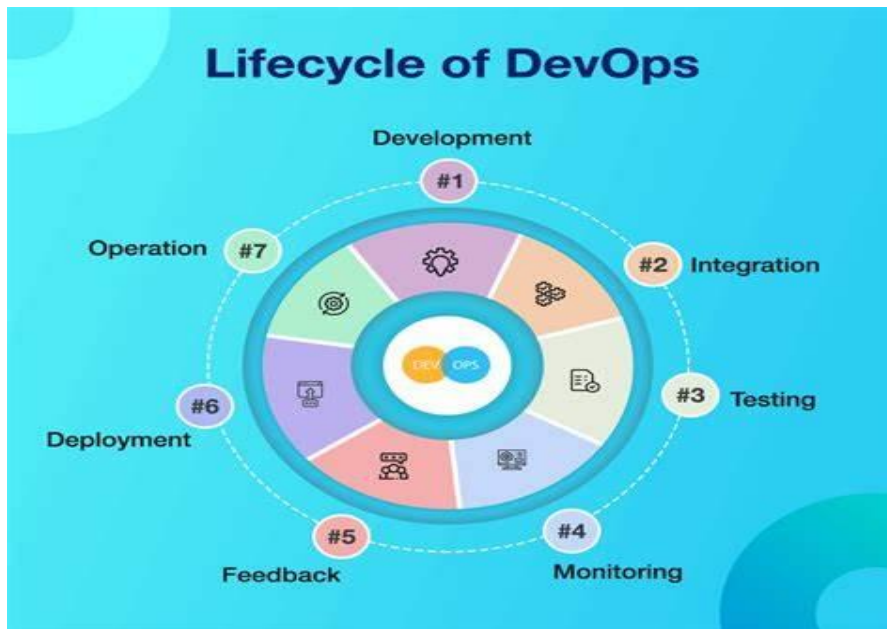
DevOps UNIT-2

DevOps Lifecycle for Business Agility

The DevOps lifecycle refers to the continuous set of activities involved in the development, deployment, and operation of software applications within a DevOps culture. It encompasses various stages, from planning and coding to testing, deployment, and monitoring. The primary goal of the DevOps lifecycle is to facilitate collaboration, streamline processes, and enable rapid and reliable delivery of software products.

Business agility, on the other hand, refers to an organization's ability to adapt and respond quickly to changing market conditions, customer needs, and emerging opportunities. It involves being flexible, innovative, and responsive in the face of uncertainty.

When we talk about the DevOps lifecycle for business agility, it means utilizing DevOps principles and practices to enhance an organization's ability to achieve business agility. By integrating DevOps into the software development and delivery processes, organizations can respond more effectively to market demands and deliver value to customers faster.



Here's how the DevOps lifecycle contributes to business agility:

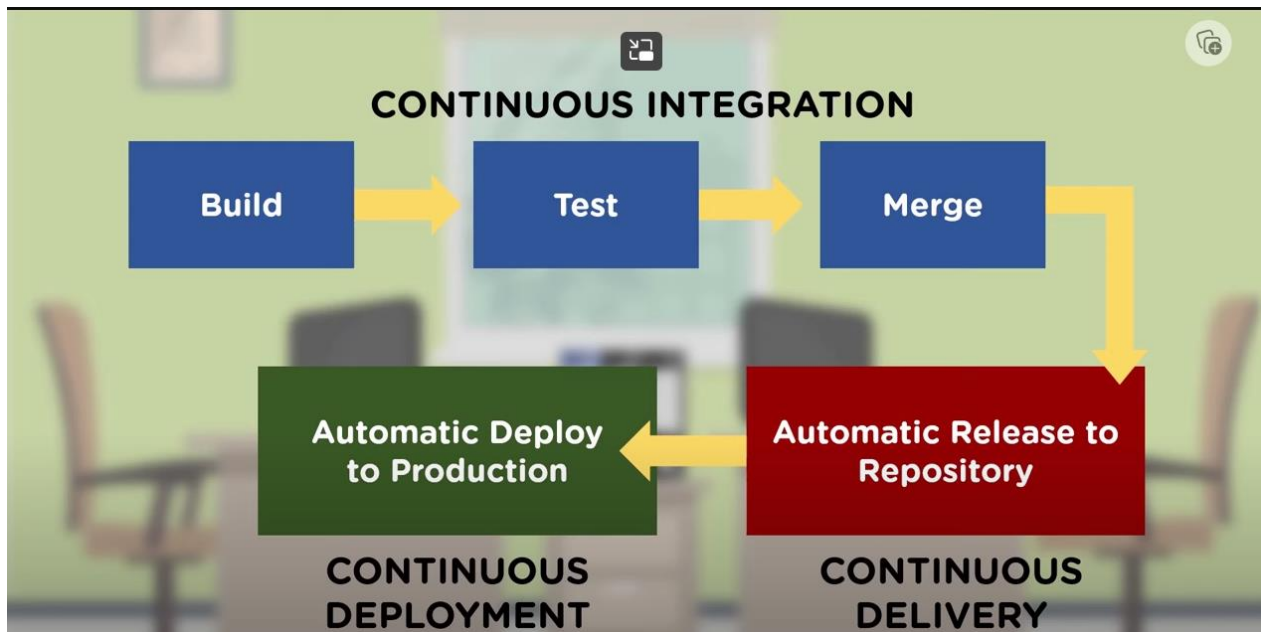
1. **Plan:** In this stage, business objectives and customer requirements are translated into actionable plans. DevOps practices, such as cross-functional collaboration and feedback loops, help ensure that the development and operations teams align their efforts with business goals, fostering agility from the outset.
2. **Develop:** This stage involves coding and building software applications. DevOps emphasizes automation, continuous integration (CI), and continuous delivery (CD), enabling rapid and efficient development cycles. By automating repetitive tasks, organizations can reduce time-to-market and respond swiftly to changing business needs.
3. **Test:** Testing is crucial to ensure software quality and reliability. DevOps encourages the use of automated testing frameworks and continuous testing practices, enabling faster feedback on application functionality, performance, and security. Rapid testing iterations help organizations adapt their products more effectively, enhancing agility.
4. **Deploy:** The deployment stage involves packaging and releasing software to production environments. DevOps promotes infrastructure

automation and deployment automation, enabling frequent and reliable deployments. By automating the deployment process, organizations can reduce deployment errors, roll back changes if necessary, and respond quickly to market demands.

5. Operate: Once software is deployed, it enters the operational phase. DevOps emphasizes monitoring and observability, enabling real-time visibility into application performance and user feedback. By monitoring key metrics and collecting customer insights, organizations can proactively identify and address issues, improving business agility through faster response times and continuous improvement.

6. Learn: The DevOps lifecycle emphasizes a culture of continuous learning and improvement. Teams collect feedback from various stages of the lifecycle, analyze data, and identify areas for optimization. By fostering a culture of learning, organizations can adapt their processes, products, and strategies based on market dynamics, customer feedback, and emerging opportunities.

By integrating DevOps practices throughout the software development and delivery lifecycle, organizations can achieve faster time-to-market, higher quality products, and increased responsiveness to changing business needs. This, in turn, enhances their overall business agility, allowing them to stay competitive and thrive in a rapidly evolving marketplace.



What is continuous testing in DevOps?

Continuous testing in DevOps is an approach to software testing that emphasizes automated testing throughout the software development and delivery process. It aims to provide fast and frequent feedback on the quality of the software, enabling rapid and reliable releases.

Traditionally, testing occurs at the end of the development cycle, often resulting in delays and bottlenecks. In contrast, continuous testing integrates testing activities into every phase of the DevOps lifecycle, from planning and development to deployment and operation. It involves automating tests and executing them continuously, allowing for early detection and resolution of issues.

Key aspects of continuous testing in DevOps include:

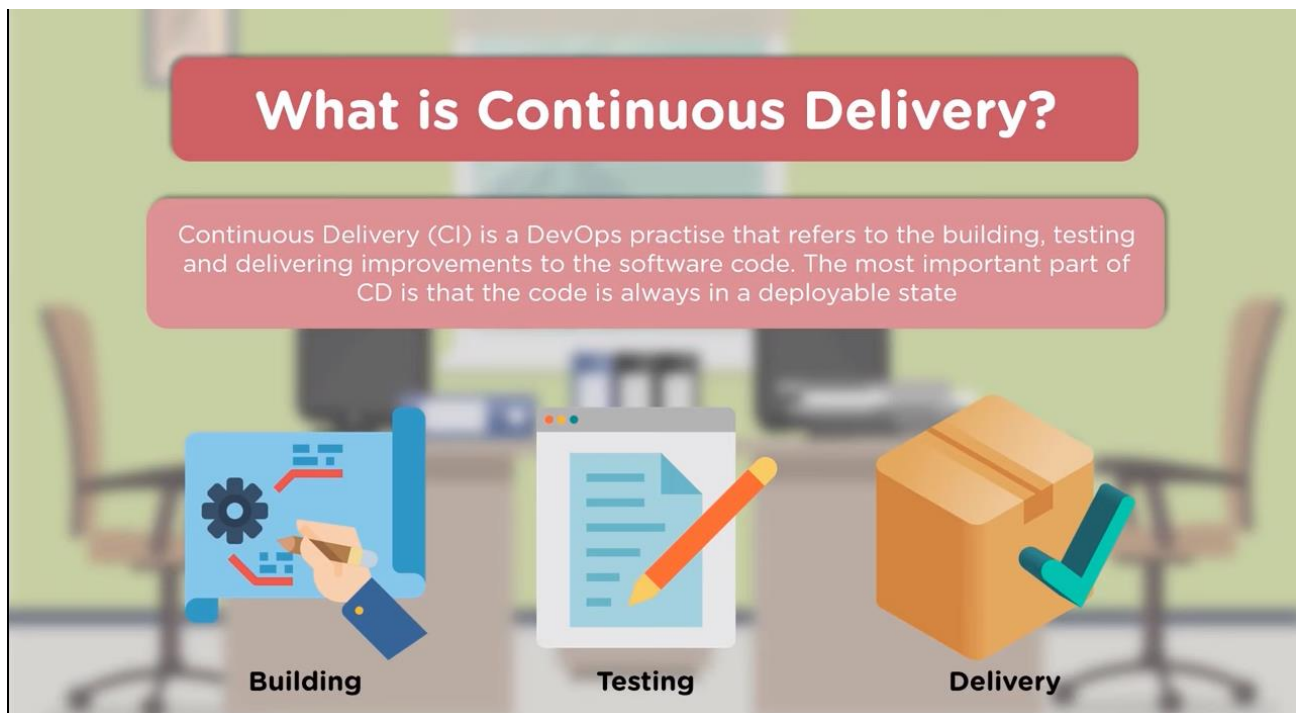
1. **Test Automation:** Continuous testing relies heavily on automation. Testing tools and frameworks are used to create automated test scripts that can be executed quickly and repeatedly. This automation reduces manual effort, accelerates testing cycles, and ensures consistent and reliable results.

2. **Early and Frequent Testing:** Continuous testing starts as early as possible in the development process. Testing is performed in parallel with development, allowing for faster feedback on the quality of code. With each code change, relevant tests are executed immediately to catch issues early and prevent them from escalating.
3. **Integration with Continuous Integration/Delivery (CI/CD):** Continuous testing seamlessly integrates with CI/CD pipelines. Automated tests are executed as part of the pipeline, triggered by code changes or scheduled intervals. This integration ensures that tests are performed consistently and automatically with each code build and deployment.
4. **Comprehensive Test Coverage:** Continuous testing aims to cover all aspects of software quality, including functional testing, performance testing, security testing, and more. It involves a mix of unit tests, integration tests, system tests, and user acceptance tests to provide comprehensive coverage of the application's behavior.
5. **Test Environment Management:** Continuous testing requires managing test environments effectively. DevOps teams create and maintain test environments that closely resemble production environments, ensuring accurate testing and reducing the risk of issues occurring in production due to environmental differences.
6. **Continuous Monitoring and Reporting:** Continuous testing involves monitoring the test execution, collecting test results, and generating reports. Real-time visibility into test results and metrics helps teams identify trends, track progress, and make data-driven decisions to improve software quality.

The benefits of continuous testing in DevOps include:

- **Faster time-to-market:** Early and continuous testing reduces the time required for testing, allowing faster release cycles and quicker delivery of new features and enhancements.
- **Higher quality software:** Continuous testing catches defects and issues early, enabling prompt remediation. This results in higher-quality software with fewer bugs and improved user satisfaction.
- **Increased confidence:** Automated tests provide consistent and reliable results, increasing confidence in the software's stability and functionality.
- **Agile adaptation:** Continuous testing facilitates rapid adaptation to changing requirements and market conditions, supporting the agility of DevOps practices.

Overall, continuous testing in DevOps helps ensure that software meets quality standards, reduces risks, and enhances the speed and agility of software delivery.

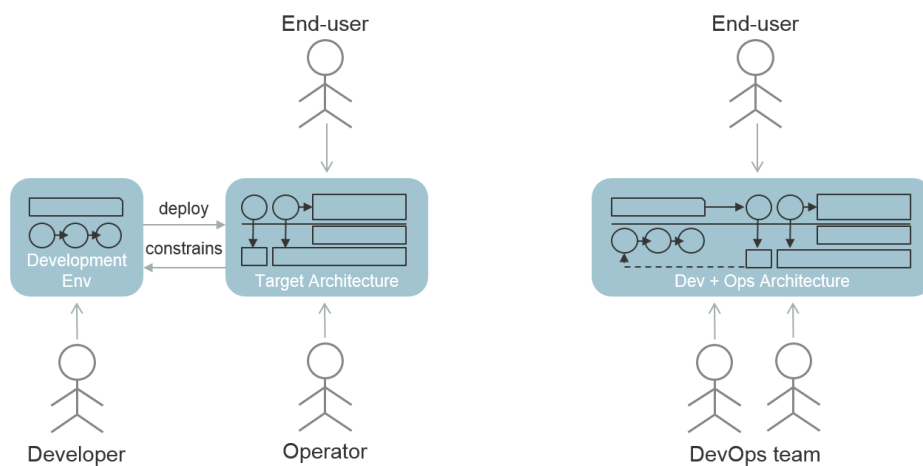


DevOps Influence on Architecture and the Monolithic Scenario

Introduction:

In today's software development landscape, DevOps practices play a crucial role in shaping the architecture of software systems. DevOps emphasizes collaboration, automation, and continuous delivery, which in turn influence the design and implementation of software architecture. This study material explores the impact of DevOps on architecture and discusses the monolithic scenario, a traditional approach to software architecture.

Architecture separated from Dev → Consolidated DevOps architecture



DevOps Influence on Architecture:

1. Collaboration:

- DevOps promotes cross-functional collaboration between development, operations, and other teams.
- Collaboration influences architectural decisions by fostering shared ownership and collective responsibility for system design.
- Early feedback and continuous improvement are encouraged throughout the development lifecycle.

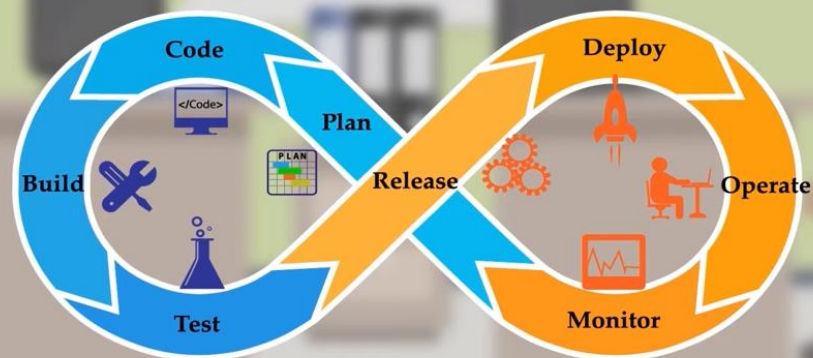
2. Continuous Integration and Deployment (CI/CD):

- DevOps emphasizes frequent integration and deployment of software changes.

- Architecture needs to support CI/CD pipelines and automation for rapid and reliable delivery.
- Components should be designed to enable independent building, testing, and deployment.

What is Continuous Integration?

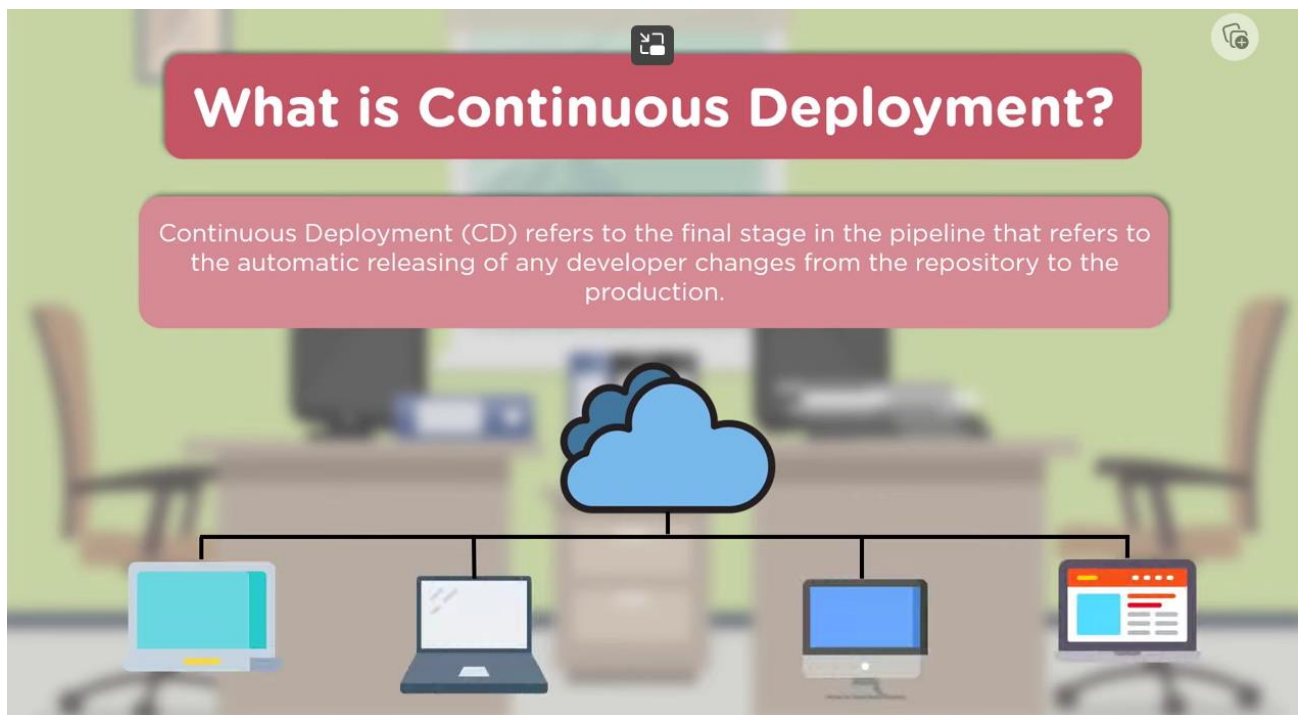
Continuous Integration (CI) is a DevOps software development practise, that enables the developers to merge their code changes in the central repository, so that automated builds and tests can be run



Why is CI/CD referred to as the best DevOps practise?

- CI and CD provides continuous feedback from the customers and also from the DevOps team. This increases the transparency of any problem in the team or outside
- The process enables faster release of the product. The failures can now be detected faster and hence fixed faster, which increases the speed of release





3. Scalability and Resilience:

- DevOps focuses on designing architectures that can scale and handle high loads effectively.
- Architectural decisions should consider horizontal scaling, load balancing, and fault tolerance mechanisms.
- Resilience should be built into the architecture to handle failures and recover quickly.

4. Infrastructure as Code (IaC):

- DevOps treats infrastructure as code, enabling automation and version control of infrastructure provisioning and configuration.
- The architecture should support IaC principles, allowing automated creation and management of infrastructure resources.

5. Observability and Monitoring:

- DevOps emphasizes monitoring and observability to gain insights into application performance and behavior.
- The architecture should facilitate the collection of relevant metrics and logs, integration with monitoring tools, and implementation of distributed tracing and logging mechanisms.

The Monolithic Scenario:

A monolithic scenario in DevOps means making and deploying the software as one big piece that works together. All the parts of the software are tightly connected and depend on each other. This can make the software fast and simple, but also hard to change and scale²⁴. If you want to update or fix one part of the software, you may have to change and redeploy the whole software². A monolithic scenario is different from a microservices-based scenario, where the software is made of many small pieces that work separately and communicate with each other¹³. A microservices-based scenario can make it easier to use DevOps practices like automation, continuous integration, and deployment strategies¹.

Conclusion:

DevOps practices help make software better by making people work together and use tools. While the monolithic scenario makes it hard to change and scale the software, DevOps principles can still help with making, testing, and deploying the software. It might be better to use other ways of making software that are more modular or microservices-based to get more benefits from DevOps practices.

Architecture Rules of Thumb:

In software architecture, "rules of thumb" are general guidelines that software architects use to make decisions about design and construction. These rules are based on experience and common sense, rather than strict mathematical formulas or scientific principles. Here are some examples of software architecture rules of thumb:

1. There is always a bottleneck ¹.
2. Your data model is linked to the scalability of your application ¹.
3. Scalability is mainly linked with cost ¹.

4. Favor systems that require little tuning to make fast ¹.
5. Use infrastructure as code ¹.
6. Use a PaaS if you're at less than 100k MAUs ¹.
7. Keep it simple ².
8. Don't repeat yourself (DRY) ².
9. Separation of concerns ³.

These rules are not meant to be followed blindly, but rather to be used as a starting point for making design decisions. Software architects use their training and experience to apply these rules in creative ways that meet the specific needs of each project.

1. The Separation of Concerns:

- This principle suggests dividing the system into distinct components or modules, each responsible for a specific concern.
- Separation of concerns improves maintainability, reusability, and testability of the system.
- It also reduces the impact of changes in one component on other parts of the system.

- 2. Modularity:

- Designing modules that are loosely coupled and highly cohesive promotes modularity.
- Modules should have clear responsibilities and well-defined interfaces.
- Modularity facilitates independent development, testing, and deployment, making the system more flexible and scalable.

3. Keep It Simple:

- Simplicity is key in software architecture.
- Avoid unnecessary complexity by favoring straightforward and understandable designs.

- Complex architectures are harder to maintain, understand, and troubleshoot.

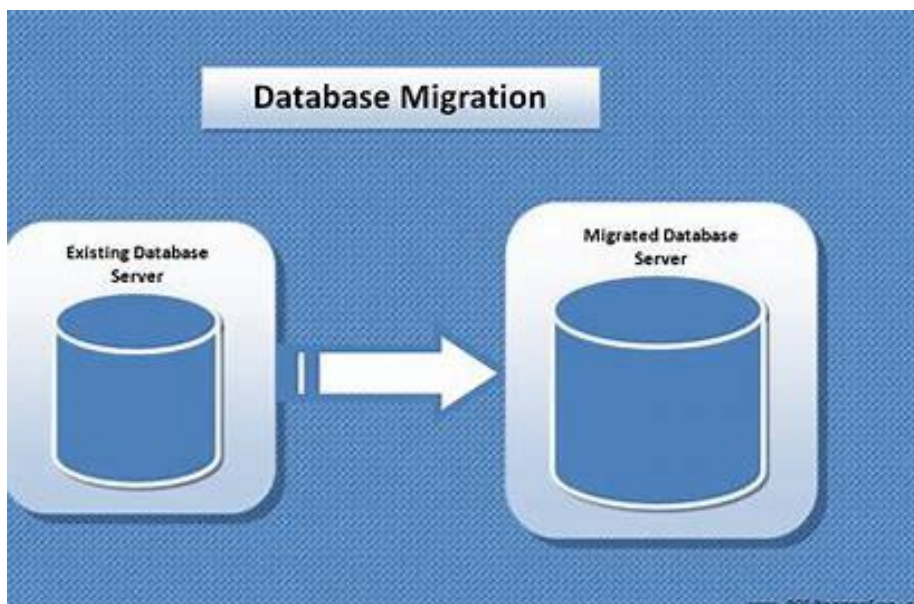
Handling Database Migrations:

1. Version Control:

- Use tools (e.g., Git) to keep track of changes to your database structure and how to move from one version to another.
- Keep your database changes together with your software code for better organization and coordination.

2. Scripted Migrations:

- Use scripts to manage your database structure changes.
 - Scripts should be able to run more than once without causing problems.
- Keep a record of what changes you have made to your database to make sure they are the same across different places where you use your system.

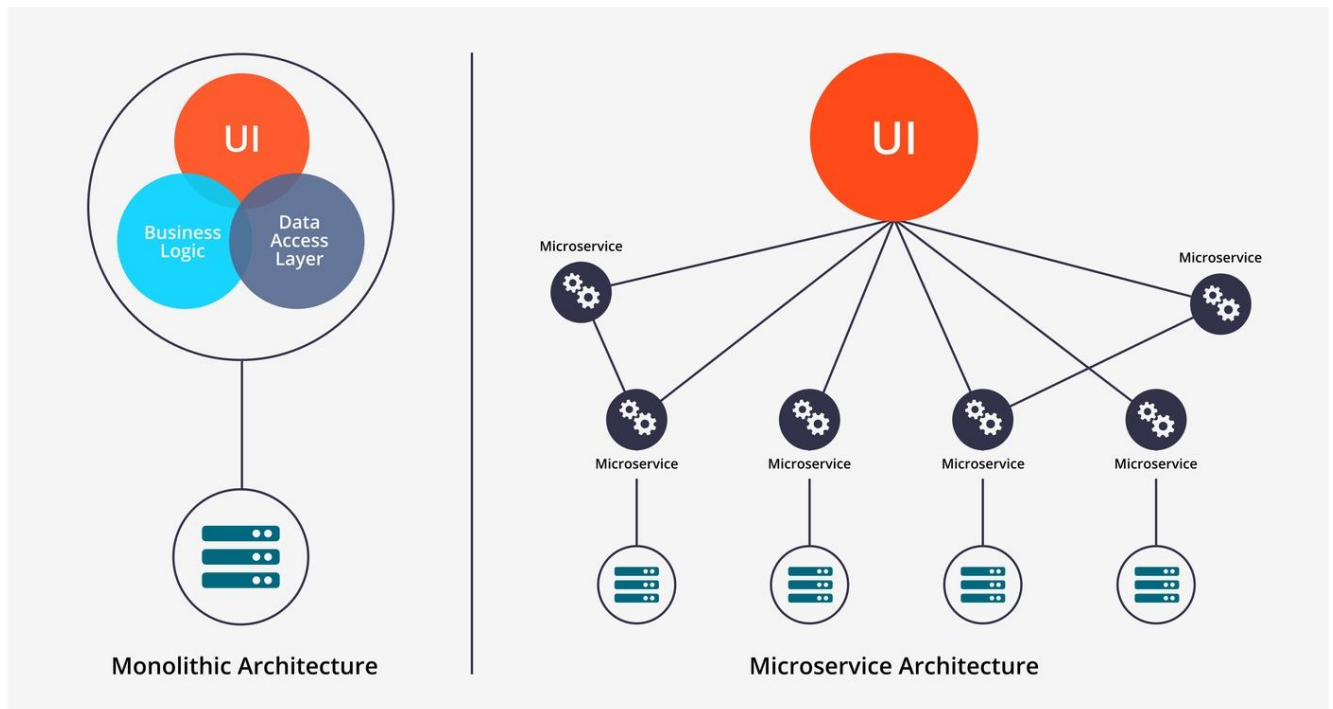


Microservices and the Data Tier:

Microservices in DevOps refer to an architectural approach where software applications are built as a collection of small, independent, and loosely-coupled services. Each service is responsible for a specific business capability and can be developed, deployed, and scaled independently. Microservices and DevOps complement each other in several ways.

Microservices is a software design approach in which an application is decomposed into a collection of loosely coupled services. Each service is self-contained and performs a single function.

Microservices and the data tier are two concepts related to software architecture and the organization of data within a microservices-based system.



1. Microservices:

- Microservices architecture is an approach where a complex software application is built as a collection of small, independent services.

Microservices is a way of making software as many small parts that do one thing only.

- Each part can be made, changed, and used by itself, making it easier to work with and change.
- Each part talks to other parts using simple ways like web or messages.
- This way of making software helps teams work faster, easier, and better.
- Each part can use its own tools, data, and ways of working.

2. The Data Tier in Microservices:

- The data tier in microservices is how you store and use your data in a microservices system.
- In a microservices system, each part has its own data that it controls and uses.
- This makes each part more independent and less dependent on other parts.
- Each part can use the best way of storing and using data for its needs (e.g., tables, documents, or memory).
- Data in a microservices system is often used and changed through simple ways provided by each part.
- Parts talk to each other about data by sharing information through events, messages, or web calls.

Considerations for the Data Tier in Microservices:

- Data consistency: Making sure data is the same across parts can be hard. You can use techniques like making data the same over time, limiting changes to one part at a time, or using events to tell other parts about changes.

- Data duplication: Each part may need to have its own copy of some data that it needs. You need to think about how to keep the copies the same and when to update them.
- Data access patterns: The way of storing and using data should match the needs and speed of each part.
- Data governance: Good practices for managing data, such as deciding who owns the data, who can use the data, and how to protect the data, are important when using data across parts.

By making the data tier separate in a microservices system, teams can work better, faster, and easier. But you need to plan and think carefully about how to make sure the data is good and works well across the system.

Conclusion: Some tips for making good software systems are breaking down your system into smaller parts that do one thing only, making parts that are not too dependent on each other and do their job well, and avoiding making your system too complicated by choosing simple and clear designs. Changing your database structure without breaking your system requires keeping track of changes and using scripts to make changes. In microservices systems, having one database for each part and using events to talk to other parts makes your system more independent and scalable. When using DevOps and architecture together, using tools to make, watch, and fix your system are important for keeping your system working well. By following these tips and concepts, you can make strong and good software systems.

DevOps, architecture, and resilience.

DevOps is a way of working together and using tools to make software faster and better. The text talks about some concepts and tips for making good software systems using DevOps. The concepts and tips are:

Architecture in DevOps:

- Architecture is how you plan and organize your software system. It includes the parts of your system, how they work together, and how your system is made.
- In DevOps, architecture is affected by the need to work together, use tools, and make software quickly and often.
- The architecture should help you follow the rules and ways of DevOps, such as making, testing, and changing your software often and easily.
- It involves making small, simple, and separate parts that can be made, tested, and changed by themselves.
- The architecture should make it easy to join the making and using of your software system, making it faster and better.

Resilience in DevOps:

- Resilience is how well your software system can handle problems, change, and recover from trouble.
- In DevOps, resilience is important to keep your system working well, fast, and stable.
- Resilience tips involve making your system so that problems don't affect it too much and you can fix them quickly.
- This includes having backup plans, ways to avoid or handle problems, and ways to keep your business going when problems happen.
- Using tools and watching your system play a big role in finding problems, doing something about them, and fixing them fast.



Intellyx

Good Technical Debt vs. Resilience Tradeoff

- Resilience discussion focuses on risk mitigation
 - Cost of mitigation vs. predicted cost of adverse event
- Do you take a shortcut on resilience or on functionality?



The DevOps Drumbeat

Intellyx

The relationship between Architecture and Resilience in DevOps:

- Architecture and resilience are related in DevOps because they affect each other.
- A good architecture thinks about resilience, thinking about ways to avoid or handle problems, change, and recover.
- Resilience needs affect architecture choices, such as choosing the best tools, ways to handle problems, and ways to watch and warn about your system.
- DevOps ways help you make and keep resilient architectures through using tools, making, testing, and changing your software often and easily, which help you recover and change quickly when problems happen.

In summary, architecture in DevOps is about planning and organizing your software system so that it helps you work together, use tools, and make software quickly and often. Resilience in DevOps is about making your system able to handle problems, change, and recover fast. Both architecture and resilience are connected and important for making and using strong, fast, and good software systems in DevOps.

Introduction to project management

Project management is a way of coordinating and tracking the development and delivery of software products that supports the DevOps model¹.

DevOps project managers need to have a deep understanding of the industry, the development process, and the skills needed to create the end product¹.

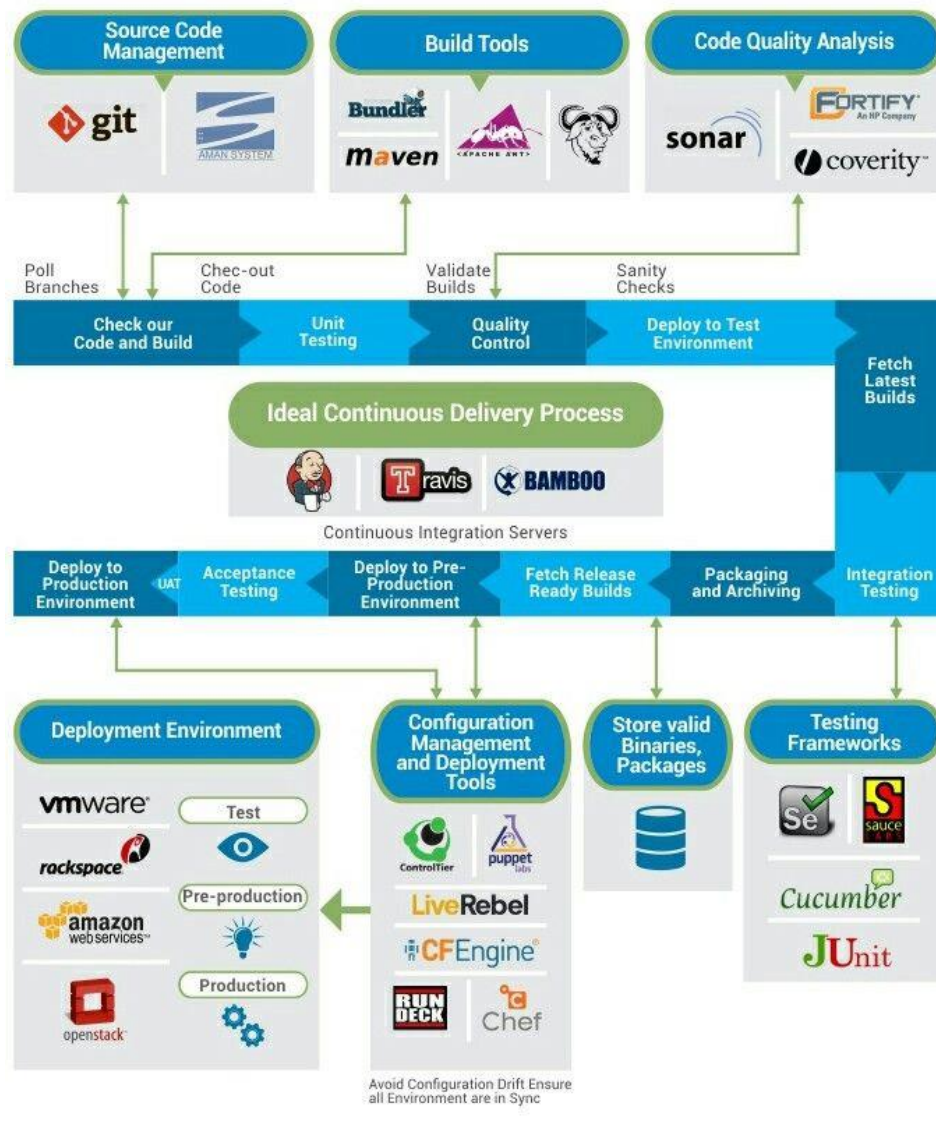
DevOps project managers also need to be aligned with the DevOps team and the client, and to resolve issues related to cost, schedule, scope, and quality¹.

DevOps project management can help create a culture of agility and change, and drive value-based behavior in the DevOps environment¹.

Project management certification can give DevOps professionals an edge in leading and overseeing the DevOps process¹.

DevOps project management is different from traditional IT project management in several ways:

- DevOps is created by one team with intimate knowledge of the product, while traditional IT is created by different teams with different standards².
- DevOps is easily understandable, while traditional IT is complex to understand².
- DevOps focuses on continuous delivery and feedback, while traditional IT focuses on monolithic and long-term initiatives³.
- DevOps bridges the gap between development and operations, while traditional IT separates them into silos³.
- DevOps embraces change and experimentation, while traditional IT resists them due to risk aversion³.



Source Code Control

Source code control (also known as version control, revision control, or source code management) is a practice that tracks and provides control over changes to computer programs, documents, or other collections of information. Source code control is a component of software configuration management¹.

Source code control enables developers to:

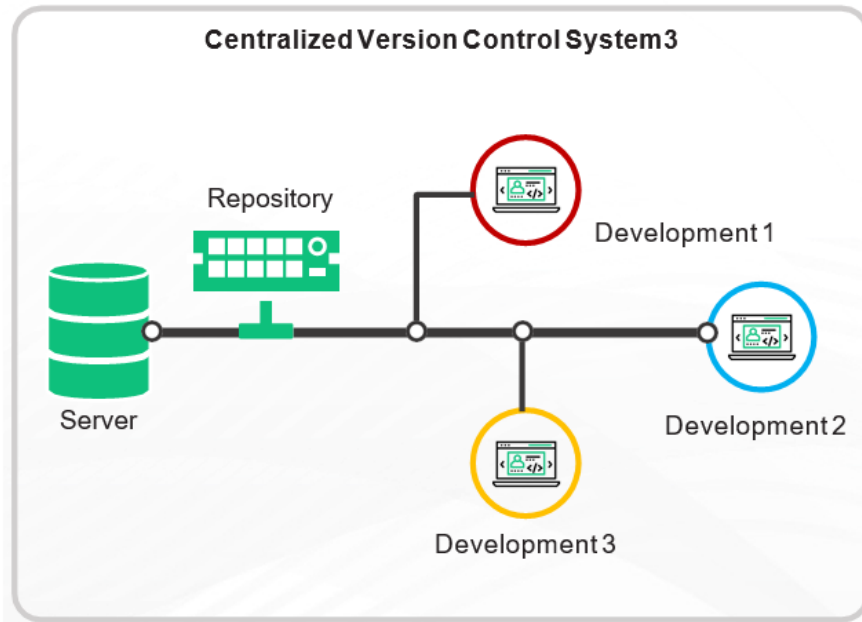
- Retrieve and run different versions of the software to locate and fix bugs
- Develop multiple versions of the software concurrently (e.g., branch and trunk)
- Compare, restore, and merge changes between revisions

- Track the history of changes and the authors of changes
- Revert a document to a previous revision



The need for source code control

- Source code control (SCC) is a system that tracks and manages changes to the source code of a software project⁴.
- SCC is needed for several reasons:
- It allows developers to work on different versions of the code without overwriting each other's changes⁴. This enables parallel development and reduces conflicts.
- It provides a history of the code changes and allows developers to revert to previous versions if needed⁴. This enables traceability and accountability.
- It facilitates collaboration and communication among developers and other stakeholders⁴. This enables transparency and feedback.
- It enables automated testing and deployment of the code⁴. This enables quality assurance and reliability.
- SCC can be implemented by various tools such as Git, Subversion, Mercurial, Perforce, etc.



The history of source code control

The need for a logical way to organize and control revisions has existed for almost as long as writing has existed, but revision control became much more important and complicated when the era of computing began. The numbering of book editions and of specification revisions are examples that date back to the print-only era¹.

The first system designed for source code control was **Source Code Control System (SCCS)**, which was started in 1972 for OS/360 by Bell Labs. SCCS was originally written in SNOBOL4 and later rewritten in C by Marc Rochkind. SCCS used binary file formats and embedded sccsid strings into source code to identify revisions².

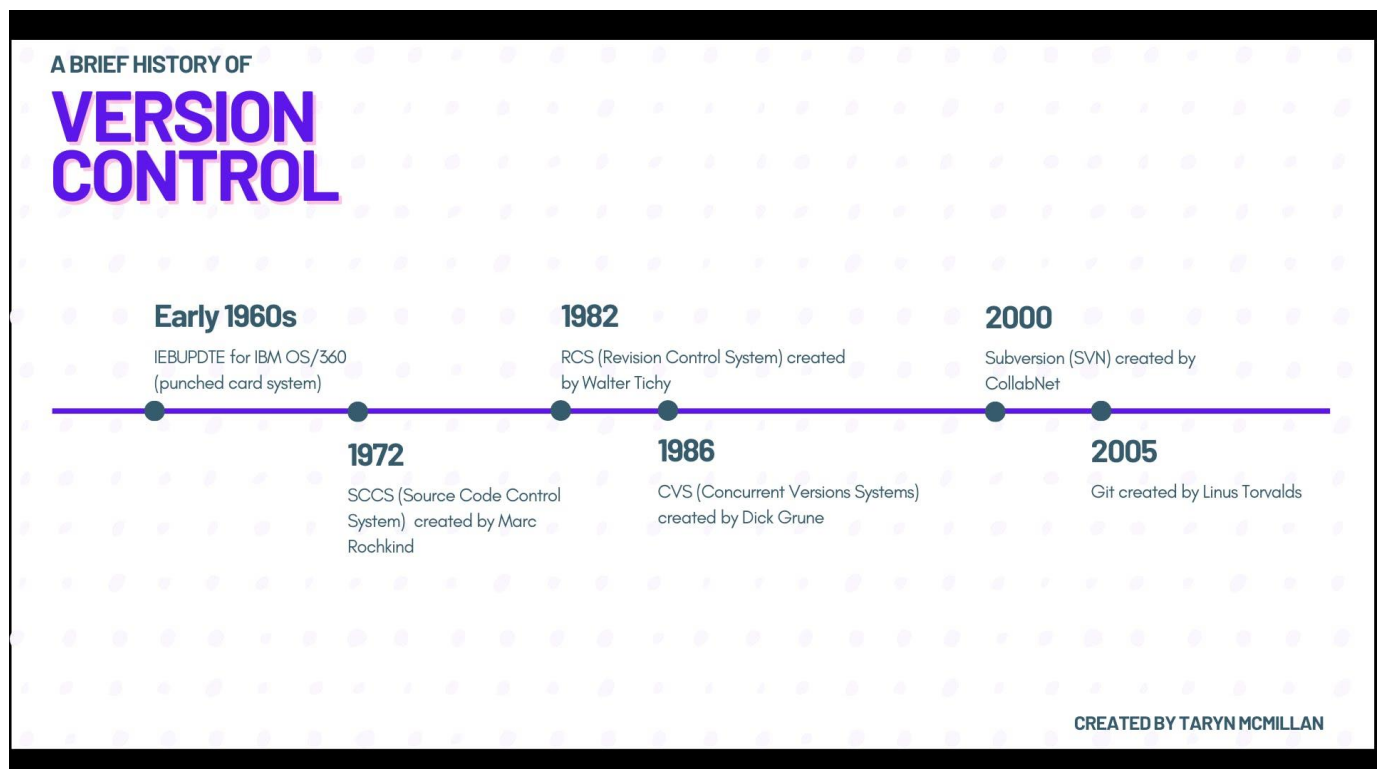
SCCS was followed by **Revision Control System (RCS)** in 1982, which was developed by Walter F. Tichy at Purdue University. RCS was a networked version of SCCS that used text-based file formats and introduced branching and merging features¹.

In 1986, **Concurrent Versions System (CVS)** was created by Dick Grune as a series of shell scripts around RCS. CVS became popular as the first widely available version control system that supported distributed, collaborative development. CVS allowed multiple developers to work on the same project simultaneously¹.

In 2000, **Subversion (SVN)** was developed by CollabNet Inc. as an attempt to create an open-source successor to CVS. SVN improved upon CVS by adding atomic commits, versioning of directories and metadata, and better handling of binary files¹.

In 2005, **Git** was created by Linus Torvalds as a distributed version control system for the Linux kernel development. Git was designed to be fast, scalable, and resilient to corruption. Git introduced the concept of local repositories that could be synchronized with remote ones¹.

Since then, many other version control systems have been developed, such as **Mercurial**, **Bazaar**, **Darcs**, **Perforce**, **BitKeeper**, etc. Some of them are centralized, some are distributed, and some are hybrid. [They differ in their features, performance, usability, and compatibility](#)¹.



Roles and code

- Roles and code are two concepts that define how developers interact with the codebase in a DevOps environment¹.

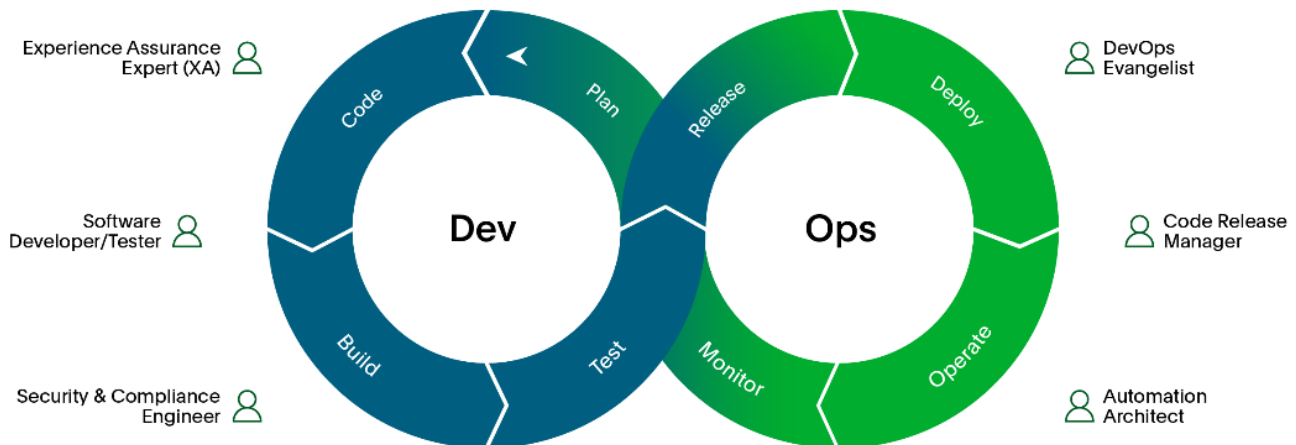
- Roles are the responsibilities and permissions that developers have in relation to the codebase. Roles can be defined by factors such as team structure, skill level, project phase, and business goals¹.
- Code is the artifact that developers produce and modify in the codebase. Code can be categorized by factors such as functionality, quality, complexity, and dependency¹.
- Roles and code can influence each other in various ways. For example:
- The complexity of the code can affect the skill level required for a role¹. For instance, a junior developer may not be able to handle complex code that requires advanced knowledge or experience.
- The functionality of the code can affect the scope and duration of a role¹. For instance, a developer who works on a core feature may have a longer-term role than a developer who works on a minor bug fix.
- The quality of the code can affect the feedback and collaboration among roles¹. For instance, a developer who writes clean and well-documented code may receive more positive feedback and collaboration from other developers than a developer who writes messy and poorly-documented code.

[Some of the key roles in a DevOps team are²³⁴⁵:](#)

- DevOps Evangelist: The leader who promotes the DevOps culture and ensures its success across the organization.
- Software Developer: The person who writes, tests, and deploys the code using various tools and methods.
- IT Engineer: The person who manages the IT infrastructure and supports the software development lifecycle.
- Systems Architect: The person who designs and optimizes the systems and platforms that host the software applications.
- Code Release Manager: The person who coordinates and tracks the code releases and deployments using agile methodologies.
- Automation Expert: The person who finds and implements the tools and processes that can automate any manual tasks in the software development lifecycle.

- Quality Assurance Engineer: The person who ensures the quality and reliability of the software applications by performing various types of testing.
- Security Engineer: The person who protects the software applications and systems from malicious attacks by performing security testing and implementing security measures.

6 essential DevOps roles



Source code management system and migrations

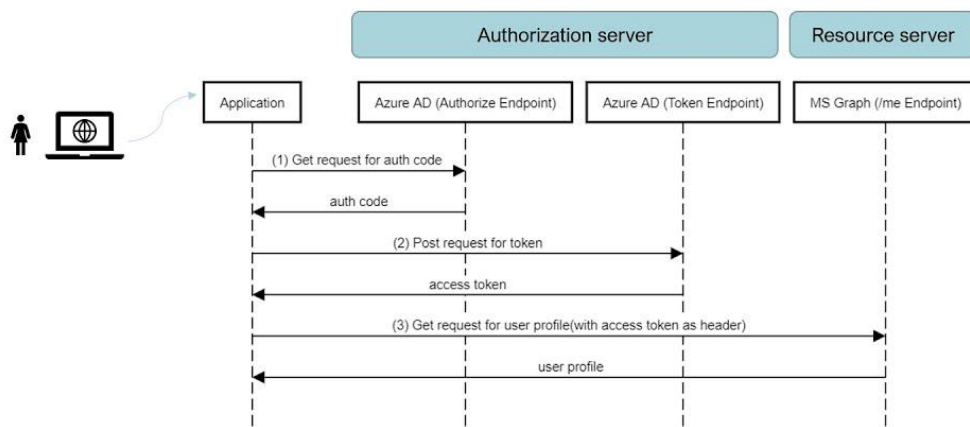
- A source code management system (SCMS) is a software tool that implements source code control functionality for a software project¹.
- Source code control is a way of keeping track of the changes that are made to the source code of a software project¹.
- A SCMS can provide features such as versioning, branching, merging, conflict resolution, access control, auditing, backup, integration, and automation¹.
- A migration is a process of moving a software project from one SCMS to another SCMS². Migrations can be motivated by factors such as performance, scalability, compatibility, security, cost, or preference².

- A migration can involve challenges such as data loss, data corruption, data inconsistency, data duplication, configuration issues, compatibility issues, learning curve, or resistance to change².
- A migration can be performed by various methods such as manual copying, scripting, importing/exporting, or using specialized tools.
- Some examples of SCMS are Git, Subversion, Mercurial, Perforce, etc¹.
- Some examples of migration tools are git-svn, git-tfs, git-p4, etc².

Shared authentication

- Shared authentication is a mechanism that allows users to access multiple services or applications with a single identity and credential.
- Shared authentication can provide benefits such as convenience, security, consistency, compliance, and personalization.
- Shared authentication can be implemented by various methods such as single sign-on (SSO), federated identity management (FIM), or social login (SL).
- SSO is a method that allows users to log in once and access multiple services or applications without logging in again. SSO can be implemented by using protocols such as OAuth or OpenID Connect.
- FIM is a method that allows users to use their identity from one service or application to access another service or application without creating a new account. FIM can be implemented by using standards such as SAML or WS-Federation.
- SL is a method that allows users to use their social media accounts to access other services or applications without creating a new account. SL can be implemented by using APIs from social media platforms such as Facebook or Twitter.

Authorization code grant flow



What is Git and GitHub explain it? How it can be used in DevOps explain?

Git is a distributed version control system that allows developers to track changes, collaborate, and manage source code efficiently. It provides a way to keep a history of code changes, enables branching and merging for parallel development, and facilitates collaboration among team members.

GitHub, on the other hand, is a web-based hosting platform for Git repositories. It adds additional features and functionality on top of Git, such as issue tracking, project management tools, and collaboration features. GitHub provides a centralized location for storing and sharing Git repositories, making it easier for teams to collaborate and contribute to projects.

In the context of DevOps, Git and GitHub play a significant role in enabling collaboration and automation in the software development and deployment processes. Here's how they are used in DevOps:

1. ****Version Control****: Git is used as a version control system to manage source code and track changes made by developers. It allows multiple developers to work on the same codebase simultaneously, preserving the integrity of the code and enabling efficient collaboration.
2. ****Code Collaboration****: Git, combined with platforms like GitHub, facilitates collaboration among developers by providing features like pull requests, code reviews, and issue tracking. Developers can propose changes, review each other's code, and discuss issues within the context of the codebase, promoting teamwork and ensuring code quality.
3. ****Continuous Integration****: Git and GitHub integrate with continuous integration (CI) tools such as Jenkins, Travis CI, or CircleCI. With CI, whenever changes are pushed to a Git repository, the CI tool automatically builds, tests, and deploys the application. This ensures that changes made by developers are continuously integrated and validated, reducing integration issues and enabling faster feedback loops.
4. ****Automated Deployments****: Git and GitHub can be used in conjunction with deployment automation tools like Ansible, Chef, or Kubernetes. These tools can pull code from Git repositories and deploy applications or infrastructure configurations automatically. This helps in achieving consistent and repeatable deployments, reducing the risk of human error and ensuring that applications are deployed in a controlled and efficient manner.

5. **Infrastructure as Code**: Git is often used to manage infrastructure code and configurations, known as Infrastructure as Code (IaC). Infrastructure code, written using tools like Terraform or CloudFormation, can be versioned and stored in Git repositories. This enables teams to track changes to infrastructure configurations, collaborate on infrastructure changes, and maintain a history of infrastructure changes over time.

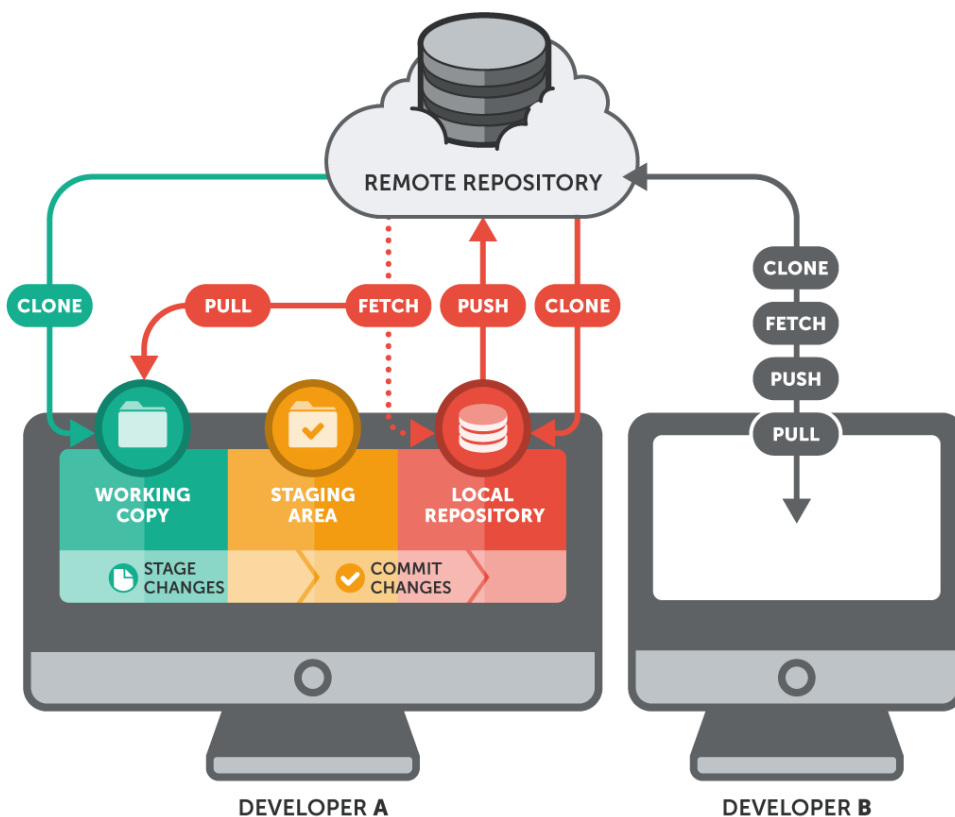
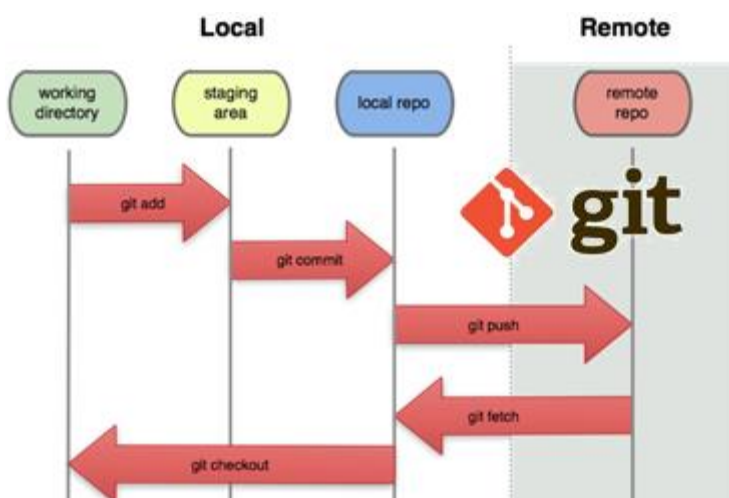
Overall, Git and GitHub provide a foundation for collaboration, version control, and automation in the DevOps lifecycle. They enable teams to work together seamlessly, manage code changes effectively, and automate the deployment process, ultimately leading to faster and more reliable software development and deployment.

Hosted Git servers

- Hosted Git servers are online platforms that provide Git hosting services for software projects.
- Hosted Git servers can offer features such as web interface, collaboration tools, issue tracking, code review, CI/CD, security measures, and cloud storage.
- Some examples of hosted Git servers are GitHub, GitLab, Bitbucket, and Azure DevOps.
- GitHub is the most popular hosted Git server that offers free public repositories and paid private repositories. GitHub also offers features such as GitHub Pages, GitHub Actions, GitHub Packages, GitHub Security Lab, and GitHub Education.
- GitLab is an open-source hosted Git server that offers free public and private repositories with unlimited collaborators. GitLab also offers features such as

GitLab CI/CD, GitLab Runner, GitLab Container Registry, GitLab Security Center, and GitLab Community Edition.

- Bitbucket is a hosted Git server that offers free private repositories for up to five users. Bitbucket also offers features such as Bitbucket Pipelines, Bitbucket Deployments, Bitbucket Code Insights, and Bitbucket Cloud.
- Azure DevOps is a hosted Git server that offers free public and private repositories for up to five users. Azure DevOps also offers features such as Azure Boards, Azure Pipelines, Azure Repos, Azure Test Plans, and Azure Artifacts.



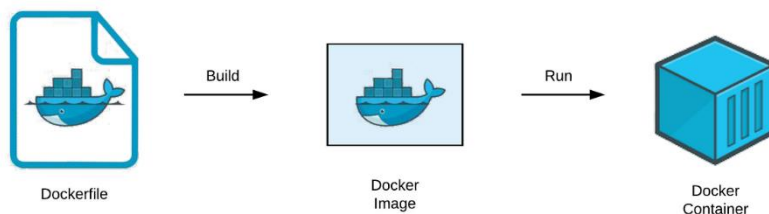
Different Git server implementations

- Git server implementations are different ways of setting up and running a Git server for a software project.
- Git server implementations can vary by factors such as hosting location, hosting provider, hosting platform, hosting service, and hosting configuration.
- Some examples of Git server implementations are:
- Self-hosted Git server: A Git server that is hosted on a local machine or a private network. This option gives the most control and customization over the Git server but also requires more maintenance and security measures. Examples of self-hosted Git servers are Gitea, Gogs, GitBucket, and Bonobo Git Server.
- Cloud-hosted Git server: A Git server that is hosted on a public cloud platform such as AWS, Azure, or Google Cloud. This option gives the most scalability and reliability for the Git server but also requires more cost and compliance measures. Examples of cloud-hosted Git servers are AWS CodeCommit, Azure DevOps Services, and Google Cloud Source Repositories.
- Hybrid-hosted Git server: A Git server that is hosted on a combination of local and cloud platforms. This option gives the best of both worlds for the Git server but also requires more integration and synchronization measures. Examples of hybrid-hosted Git servers are GitHub Enterprise Server, GitLab Self-Managed, and Bitbucket Server.

Docker intermission

- Docker is a software platform that enables developers to build, run, and share applications using containers.
- Containers are isolated environments that package the application code and its dependencies in a standardized format.

- Docker can help DevOps teams to achieve faster and more reliable software delivery by providing benefits such as portability, scalability, consistency, security, and efficiency.
- Docker can be used by various tools such as Docker Engine, Docker Compose, Docker Swarm, Docker Hub, and Docker Desktop.
- Docker Engine is the core component of Docker that creates and runs containers.
- Docker Compose is a tool that defines and runs multi-container applications using a YAML file.
- Docker Swarm is a tool that orchestrates and manages a cluster of Docker nodes.
- Docker Hub is a service that hosts and distributes container images.
- Docker Desktop is a tool that allows developers to use Docker on their local machines.



Gerrit

- Gerrit is a web-based code review and project management tool for Git projects.
- Gerrit can help DevOps teams to improve the quality and collaboration of their code by providing features such as code review, code approval, code merging, code branching, code testing, code integration, and code deployment.

- Gerrit can be integrated with various tools such as Jenkins, Jira, Eclipse, and IntelliJ IDEA.
- Jenkins is a CI/CD tool that automates the building, testing, and deploying of the code.
- Jira is an issue tracking and project management tool that manages the tasks and workflows of the project.
- Eclipse and IntelliJ IDEA are integrated development environments (IDEs) that support editing, debugging, and refactoring of the code.

The pull request model

- The pull request model is a workflow for Git projects that involves creating and reviewing changes to the codebase before merging them into the main branch.
- The pull request model can help DevOps teams to achieve better code quality and collaboration by providing benefits such as peer review, feedback, discussion, validation, and traceability.
- The pull request model can be implemented by various tools such as GitHub, GitLab, Bitbucket, or Azure DevOps.
- GitHub is a hosted Git server that offers free public repositories and paid private repositories. GitHub also offers features such as GitHub Pages, GitHub Actions, GitHub Packages, GitHub Security Lab, and GitHub Education.
- GitLab is an open-source hosted Git server that offers free public and private repositories with unlimited collaborators. GitLab also offers features such as GitLab CI/CD, GitLab Runner, GitLab Container Registry, GitLab Security Center, and GitLab Community Edition.
- Bitbucket is a hosted Git server that offers free private repositories for up to five users. Bitbucket also offers features such as Bitbucket Pipelines, Bitbucket Deployments, Bitbucket Code Insights, and Bitbucket Cloud.

- Azure DevOps is a hosted Git server that offers free public and private repositories for up to five users. Azure DevOps also offers features such as Azure Boards, Azure Pipelines, Azure Repos, Azure Test Plans, and Azure Artifacts.

GitLab

- GitLab is a web-based platform that provides a complete DevOps lifecycle toolchain for Git projects.
- GitLab can help DevOps teams to deliver software faster and more reliably by providing features such as source code management, project management, issue tracking, code review, CI/CD, security testing, monitoring, and analytics.
- GitLab can be used by various tools such as GitLab Runner, GitLab Pages, GitLab Container Registry, GitLab Security Center, and GitLab Community Edition.
- GitLab Runner is a tool that executes the jobs defined in the `.gitlab-ci.yml` file on the GitLab CI/CD pipeline.
- GitLab Pages is a feature that allows users to create static websites from their GitLab repositories.
- GitLab Container Registry is a feature that allows users to store and manage their container images on GitLab.
- GitLab Security Center is a feature that allows users to scan their code for vulnerabilities and compliance issues on GitLab.
- GitLab Community Edition is an open-source version of GitLab that offers most of the features of GitLab for free.

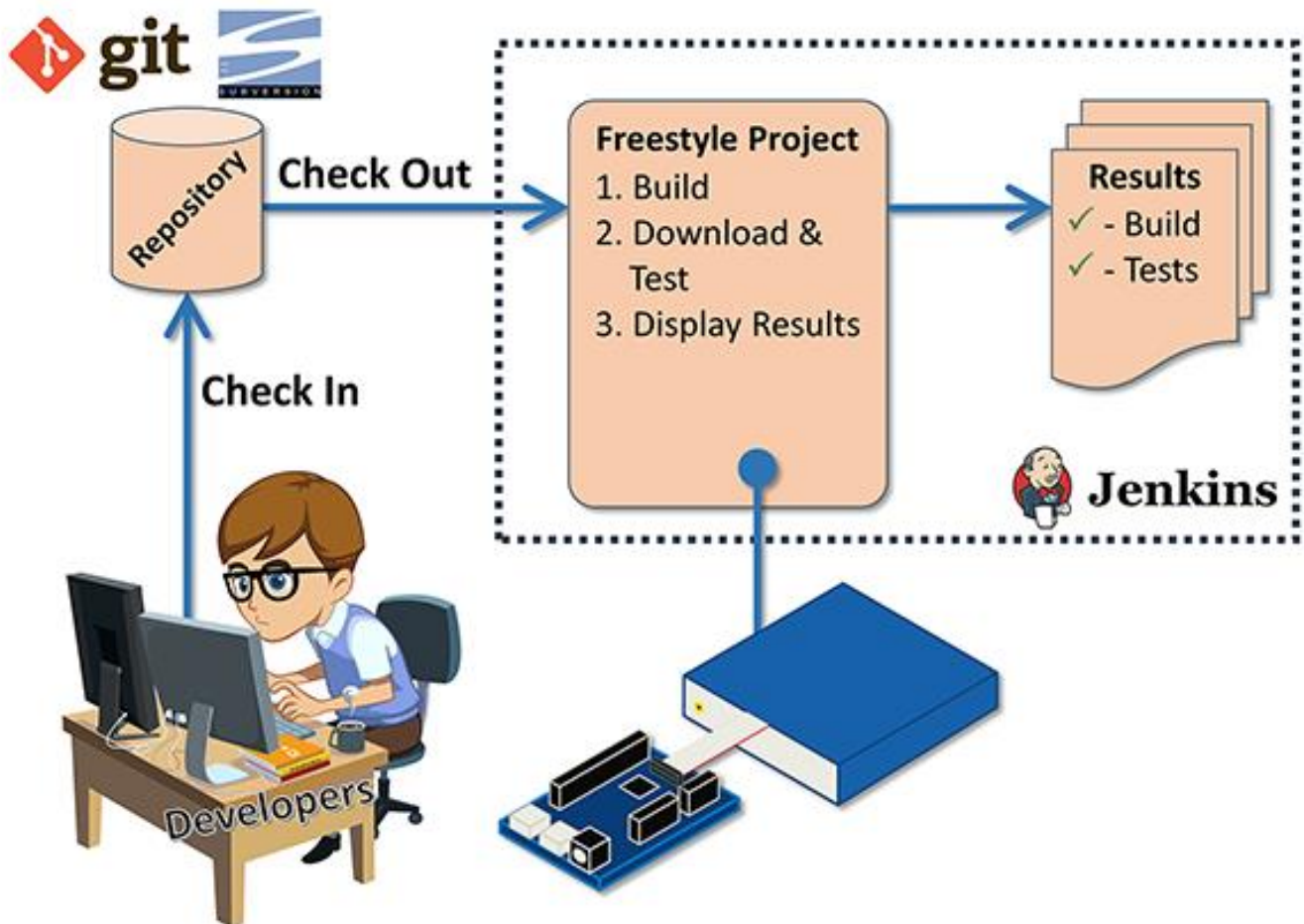
DevOps UNIT-4

Build Systems

A build system is a software tool that automates the process of building software. It takes source code as input and produces an executable binary as output. Build systems are essential for DevOps because they allow teams to automate the process of building and deploying software, which can help to improve the speed, reliability, and efficiency of the development process.

There are a number of different build systems available, each with its own strengths and weaknesses. Some popular build systems include:

- **Jenkins:** Jenkins is a popular open-source build server that is used by many DevOps teams. It is a flexible and extensible platform that can be used to automate a wide variety of tasks, including building, testing, and deploying software.
- **Gradle:** Gradle is a build automation tool that is used to build and manage Java-based projects. Gradle is a powerful and flexible tool that can be used to automate a wide variety of build tasks.
- **Maven:** Maven is a build automation tool that is used to build and manage Java-based projects. Maven is a popular choice for Java projects because it is easy to use and it provides a number of features that are specifically designed for Java projects.



What are the benefits of using a build system?

There are a number of benefits to using a build system, including:

- * **Increased speed:** Build systems can automate the process of building software, which can save time and improve the speed of the development process.
- * **Improved reliability:** Build systems can help to ensure that the build process is repeatable and reliable. This can help to reduce the number of errors and bugs in the software.
- * **Enhanced efficiency:** Build systems can help to improve the efficiency of the development process by automating tasks such as dependency management and code formatting.
- * **Improved collaboration:** Build systems can help to improve collaboration between developers by providing a central place to store the build process and the build artifacts.

what is Jenkins and what is Jenkins build server?

Jenkins is an open-source automation server that facilitates continuous integration (CI) and continuous delivery (CD) processes. It provides a platform for automating various tasks involved in software development, such as building, testing, and deploying applications.

Jenkins is primarily known for its role as a build server. A build server is a software tool or server that automates the process of building software from source code. Jenkins acts as a central hub that orchestrates the build process, integrating with version control systems, triggering builds based on defined events, managing dependencies, executing build scripts, and generating build artifacts.

Jenkins is an open-source automation server that helps automate the process of building, testing, and deploying software. It is a flexible and extensible platform that can be used to automate a wide variety of tasks.

A Jenkins build server is a physical or virtual machine that runs the Jenkins automation server. The build server is responsible for executing the build jobs that are defined in the Jenkins configuration.

Jenkins is a popular tool for DevOps teams because it can help to improve the speed, reliability, and efficiency of the development process. It can also be used to automate a wide variety of tasks, such as:

- Building software
- Running unit tests
- Deploying software
- Managing dependencies
- Ensuring code quality
- Monitoring performance

As a build server, Jenkins offers the following key features:

1. **Build Automation**: Jenkins automates the build process by pulling the latest code changes from a version control system, compiling the source code, running tests, and creating executable artifacts.
2. **Continuous Integration**: Jenkins supports continuous integration by automatically triggering builds whenever code changes are committed to the version control system. It helps detect integration issues early and ensures that the software remains in a releasable state.
3. **Plugin Ecosystem**: Jenkins has a vast ecosystem of plugins that extend its functionality. These plugins enable integration with various tools, including version control systems, testing frameworks, code analysis tools, deployment tools, and more. They provide flexibility and customization options to adapt Jenkins to specific project requirements.
4. **Build Pipelines**: Jenkins allows the creation of build pipelines, which are sequences of interconnected build stages or jobs. Build pipelines provide a visual representation of the entire build and deployment process, enabling the execution of complex workflows, parallelization of tasks, and easy monitoring of the build progress.
5. **Distributed Build Execution**: Jenkins can distribute build tasks across multiple machines or build agents called "build slaves." This allows parallel execution of builds, increases build throughput, and supports scaling for larger projects or organizations.
6. **Extensibility and Customization**: Jenkins is highly customizable and extensible. It offers a wide range of configuration options, build triggers, and build steps, allowing users to tailor the build process to their specific needs. Custom scripts and plugins can be developed to integrate Jenkins with other tools and services.

Overall, Jenkins build server simplifies the process of building, testing, and deploying software by automating repetitive tasks, enabling continuous integration, and providing a flexible and extensible platform for integrating various tools and technologies in the software development lifecycle.

Managing build dependencies:

What are build dependencies?

Build dependencies are the libraries and software that are required to build a software project. When you build a software project, you need to make sure that all of the required dependencies are available. If a dependency is missing, the build process will fail.

Why is it important to manage build dependencies?

Managing build dependencies is important for a number of reasons, including:

- To ensure that the build process is repeatable: If you don't manage your dependencies, it can be difficult to reproduce the build process on another machine. This can make it difficult to debug problems and to deploy the application to production.
- To avoid build errors: If a dependency is missing, the build process will fail. This can waste time and resources, and it can also delay the release of the application.
- To improve the performance of the build process: If you manage your dependencies, you can cache the dependencies and reuse them in subsequent builds. This can improve the performance of the build process and reduce the amount of time it takes to build the application.

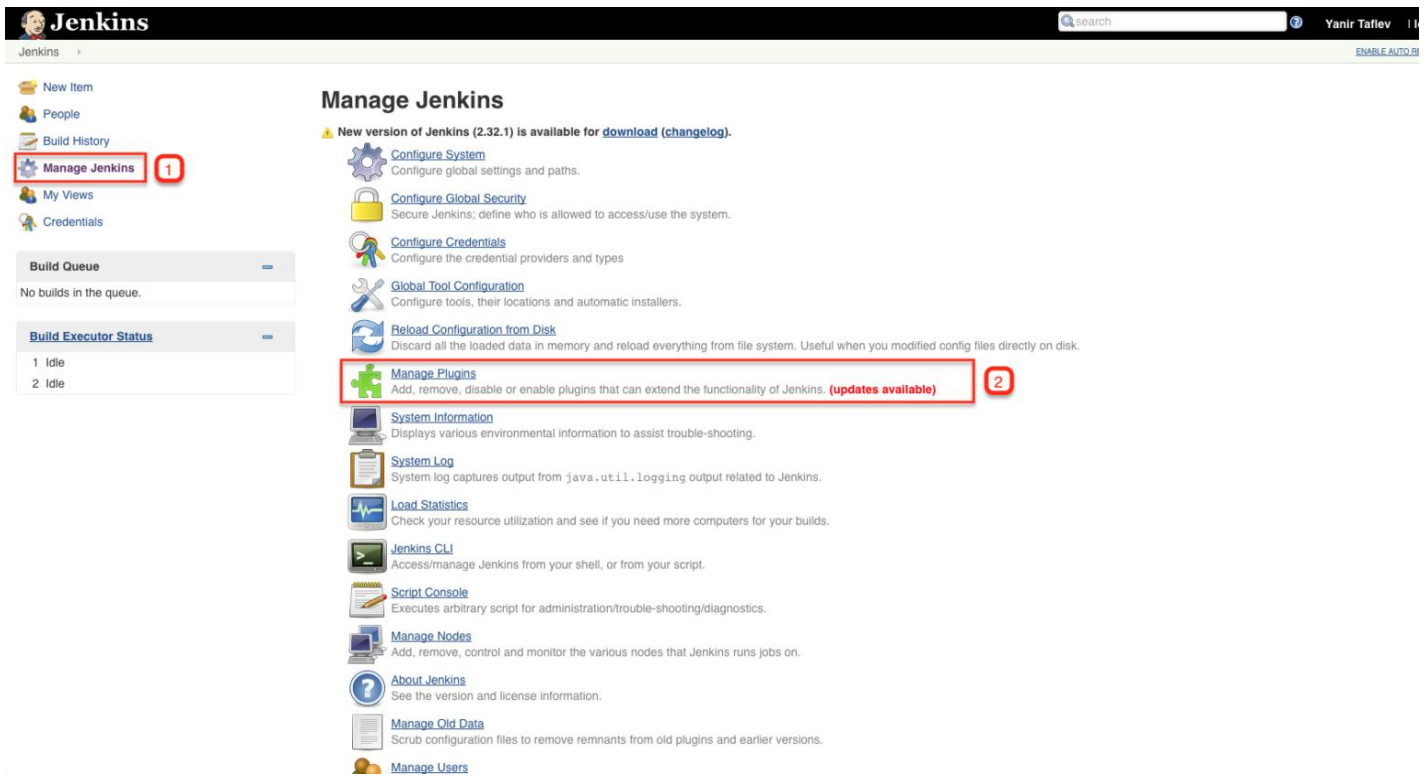
How to manage build dependencies?

There are a number of tools that can be used to manage build dependencies, including:

- **Maven:** Maven is a popular tool for managing build dependencies in Java projects. Maven uses a concept called "dependency management" to ensure that all of the required dependencies are available when the build is executed. Maven also provides a number of other features that can be used to automate the build process.
- **Gradle:** Gradle is another popular tool for managing build dependencies. Gradle is a more flexible tool than Maven, and it can be used to build a wider variety of projects. Gradle also provides a number of features that are not available in Maven, such as the ability to run parallel builds and to cache dependencies.
- **Pipenv:** Pipenv is a newer tool for managing build dependencies. Pipenv is designed for Python projects, and it provides a number of features that are not available in Maven or Gradle, such as the ability to install dependencies from PyPI and to create virtual environments.

Conclusion

Managing build dependencies is an important part of the software development process. By using a tool to manage your dependencies, you can ensure that the build process is repeatable, avoid build errors, and improve the performance of the build process.



Jenkins Plugins:

Jenkins provides a vast array of plugins that extend its functionality and integration capabilities. These plugins can be installed and configured within Jenkins to enhance its capabilities and integrate it with various tools and services. Some examples of Jenkins plugins include:

1. ****Version Control System (VCS) Plugins****: These plugins allow Jenkins to integrate with different version control systems like Git, Subversion, Mercurial, and more. They enable Jenkins to automatically trigger builds based on code changes and fetch the latest source code from the VCS.
2. ****Build Tool Plugins****: Jenkins supports integration with build tools such as Maven, Gradle, Ant, and others. These plugins provide seamless integration with these tools, allowing Jenkins to execute build scripts, manage dependencies, and generate build artifacts.
3. ****Testing Framework Plugins****: Jenkins plugins integrate with popular testing frameworks like JUnit, NUnit, Selenium, and others. They enable Jenkins to execute automated tests, collect test results, and generate test reports, providing visibility into the test coverage and overall test quality.

4. **Deployment Plugins**: These plugins help automate deployment processes by integrating Jenkins with deployment tools like Ansible, Chef, Puppet, or Kubernetes. They enable Jenkins to deploy applications to various environments, manage infrastructure configurations, and streamline the deployment pipeline.
5. **Code Analysis and Quality Plugins**: Jenkins offers plugins that integrate with code analysis tools such as SonarQube, Checkstyle, PMD, and others. These plugins analyze code quality, identify code smells, bugs, and security vulnerabilities, and provide reports and metrics for code quality improvement.
6. **Notification Plugins**: Jenkins can send notifications and alerts to team members or external systems via plugins. These plugins can integrate with various communication channels like email, Slack, Microsoft Teams, or chat platforms, ensuring timely communication of build status and alerts.

File System Layout:

The file system layout of Jenkins refers to how files and directories are organized within a Jenkins installation. Here are some key directories and their purposes:

1. **Home Directory**: The home directory is the root directory of the Jenkins installation. It contains all the Jenkins configuration files, plugins, job configurations, and other data related to the Jenkins instance.
2. **Workspace Directory**: Each Jenkins job has its own workspace directory where the source code, build artifacts, and temporary files related to the job are stored. Jenkins creates a separate workspace for each job to ensure isolation and reproducibility.
3. **Job Directories**: Inside the home directory, there is a directory for each Jenkins job. These directories store the job-specific configuration files, build history, logs, and archived artifacts.
4. **Plugin Directory**: Jenkins maintains a directory to store all the installed plugins. Each plugin has its own directory within the plugins directory, containing the plugin files and resources.

5. ****Log Directory****: Jenkins logs are stored in a separate directory. These logs provide information about the build process, job execution, errors, and other important events.

Understanding the file system layout is essential for managing Jenkins configurations, backing up critical data, troubleshooting issues, and customizing Jenkins behavior through configuration files and scripts.

The file system layout of a Jenkins project is important for ensuring that the build process runs smoothly. The project files should be organized in a way that makes sense and that is easy to understand.

The following is a recommended file system layout for a Jenkins project:

```
project/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/example/project/
│   │   │       └── App.java
│   │   └── resources/
│   └── test/
│       ├── java/
│       │   └── com/example/project/
│       │       └── AppTest.java
│       └── resources/
└── jenkins/
    ├── jobs/
    │   └── my-project/
    │       ├── config.xml
    │       └── Jenkinsfile
    └── plugins/
```


****The host server**:**

1. The host server is the machine where Jenkins is installed and runs as a service.
2. It provides the computing resources and environment for executing build tasks.
3. The host server needs to meet the hardware and software requirements for Jenkins installation.
4. It should have a stable network connection to communicate with build slaves and external systems.
5. The host server can be a dedicated physical machine, a virtual machine, or a cloud-based instance.
6. Proper security measures should be implemented to protect the host server from unauthorized access.
7. Regular maintenance and monitoring of the host server are necessary to ensure its performance and availability.
8. The host server should have sufficient storage capacity to store build artifacts and logs.
9. Backup and disaster recovery plans should be in place to safeguard the host server and its data.
10. The scalability of the host server should be considered to handle increased build load as the project grows.

****Build slaves**:**

1. Build slaves are additional machines connected to the host server to execute build tasks delegated by Jenkins.
2. They help distribute the workload and enable parallel execution of builds.
3. Build slaves can be physical machines, virtual machines, or containers.
4. Multiple build slaves can be configured to handle different operating systems, hardware architectures, or specialized build environments.
5. Build slaves need to have the necessary software and dependencies installed to execute build tasks.
6. They can be located on the same network as the host server or in remote locations.
7. Security measures, such as authentication and access control, should be implemented for build slaves.

8. Build slaves can be dynamically provisioned and released based on the build workload.
9. Monitoring the performance and availability of build slaves is important to ensure efficient build execution.
10. Regular maintenance and updates are necessary for build slaves to keep them in a healthy state.

****Software on the host**:**

1. The host server should have the required software installed, such as Java Development Kit (JDK) or Java Runtime Environment (JRE).
2. Git or other version control systems may be installed for source code management.
3. Build tools like Maven, Gradle, or Ant may be installed for compiling and packaging the code.
4. Jenkins plugins may be installed to enhance the capabilities of the host server.
5. Additional software, such as testing frameworks, code analysis tools, or deployment tools, may be installed depending on the project requirements.
6. The software on the host should be properly configured and updated to ensure compatibility and security.
7. Documentation and guidelines should be provided for installing and configuring the required software on the host.
8. Regular monitoring and maintenance of the software on the host are necessary to ensure its proper functioning.
9. Proper security measures should be implemented, including access control and vulnerability management, for the software on the host.
10. Documentation should be maintained to keep track of the installed software versions and dependencies.

****Triggers**:**

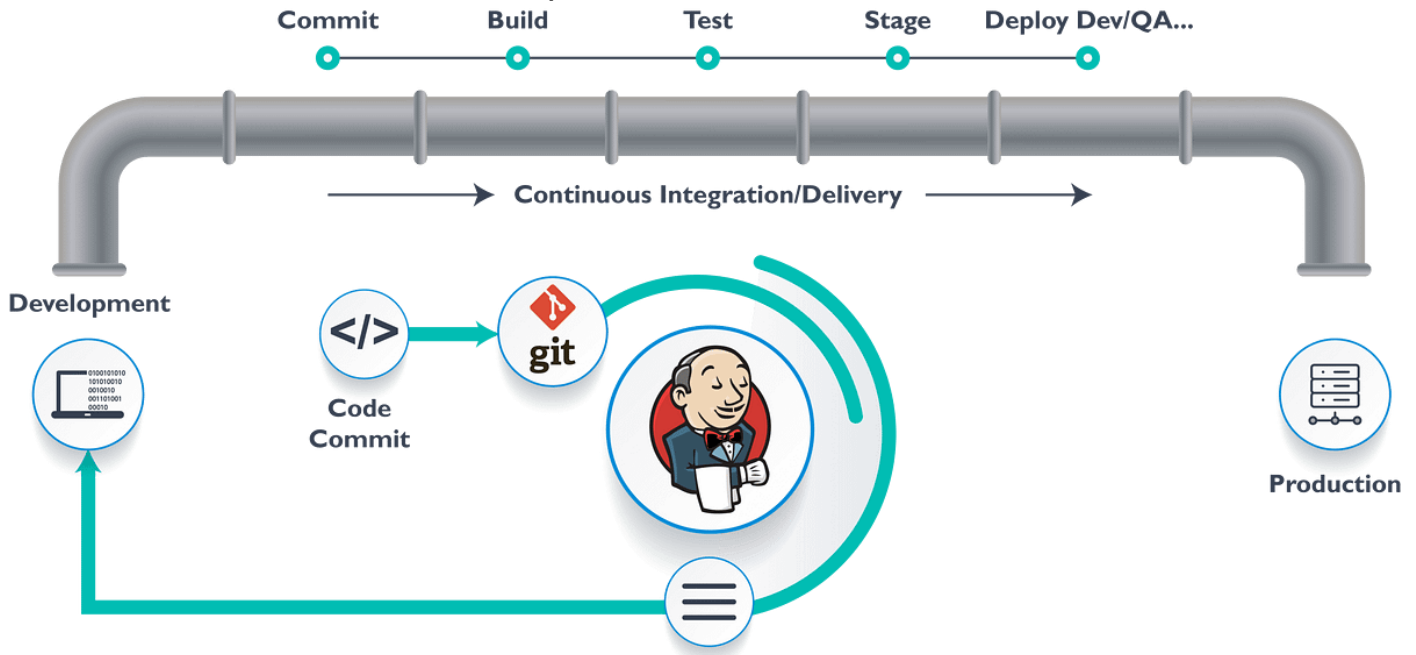
1. Triggers are events or conditions that initiate a build in Jenkins.
2. Code changes in a version control system, such as a commit or a pull request, can trigger a build.
3. Scheduled triggers allow builds to occur at specific times or intervals.
4. Manual triggers can be used to initiate a build manually by a user or an external system.

5. Trigger conditions can be configured based on specific criteria, such as branch names, file changes, or commit messages.
6. Webhooks or post-commit hooks can be used to automatically trigger builds when code changes occur.
7. Triggers can be customized and combined to create complex build workflows and ensure timely and efficient builds.
8. Care should be taken to avoid unnecessary or frequent triggers that may impact the build server's performance.
9. Monitoring and logging triggers can help track build activities and identify any issues or failures.
10. Documentation and communication should be provided to inform team members about the triggers and their configurations.

****Job chaining and build pipelines**:**

1. Job chaining is a mechanism in Jenkins to define a sequence of build jobs that are executed one after another.
2. Build pipelines allow the visualization and orchestration of the entire build and deployment process.
3. Each job in a pipeline can have its own configuration and can be triggered based on specific conditions or dependencies.
4. Pipelines provide a clear overview of the build progress and enable easy monitoring and troubleshooting.
5. Parallel execution of jobs within a pipeline can be utilized to optimize build time and resource utilization.
6. Jobs in a pipeline can be connected with various build steps, such as building, testing, packaging, and deploying.
7. Pipelines can incorporate conditional logic, error handling, and notifications to ensure smooth and reliable build processes.
8. Pipeline scripts or declarative pipeline syntax can be used to define and manage complex build workflows.
9. Visualizations, such as Jenkins Blue Ocean, can provide a more intuitive and user-friendly representation of build pipelines.

10. Regular monitoring and optimization of pipelines are important to ensure efficient and reliable build processes.



****Build servers and infrastructure as code**:**

1. Build servers are central components of the CI/CD infrastructure that automate the build process.
2. They provide a platform for executing build tasks, managing dependencies, and generating build artifacts.
3. Infrastructure as Code (IaC) refers to managing and provisioning the build infrastructure using code and configuration files.
4. IaC enables version control, reproducibility, and scalability of the build infrastructure.
5. Tools like Jenkins Configuration as Code (JCasC) allow defining and managing Jenkins configurations as code.
6. Infrastructure automation tools like Ansible, Terraform, or Chef can be used to provision and configure the build servers.
7. IaC facilitates consistency and reduces manual effort in setting up and maintaining the build infrastructure.
8. Changes to the build infrastructure can be easily managed and deployed through code changes and automated pipelines.

9. Proper testing and validation of the infrastructure code are necessary to ensure its reliability and correctness.
10. Documentation and versioning of infrastructure code and configurations are important for maintaining a stable and manageable build environment.

Infrastructure as Code workflow



“It’s all software”



****Building by dependency order**:**

1. Building by dependency order is a technique of ordering build jobs based on their dependencies.
2. It ensures that the dependent modules, libraries, or components are built and available before building the dependent projects.
3. Dependencies can be defined at the source code level, such as through project references or import statements.
4. Jenkins can analyze these dependencies and determine the order in which the jobs should be executed.
5. Properly managing build dependencies helps avoid build failures due to missing or outdated components.
6. Tools like Maven or Gradle provide dependency management capabilities to automatically resolve and download required libraries.
7. Care should be taken to ensure that the build process considers both direct and transitive dependencies.

8. Dependency graphs or visualizations can assist in understanding and managing the complex interdependencies of the build.
9. Regular updates and maintenance of dependencies are important to keep the build process up to date and secure.
10. Documentation and communication should be provided to inform developers about the dependency order and any changes in the dependencies.

****Build phases**:**

1. Build phases represent different stages or steps in the build process.
2. Common build phases include compilation, testing, packaging, deployment, and post-build actions.
3. Each build phase performs specific tasks and contributes to the overall build process.
4. Build phases can be defined and customized based on the project's requirements and development practices.
5. Proper sequencing and orchestration of build phases are essential for a successful and efficient build.
6. Tools like Maven or Gradle provide predefined build phases and hooks to execute specific tasks at different stages.
7. Monitoring and logging of build phases help track the progress and identify any failures or bottlenecks.
8. Integration with testing frameworks and code analysis tools can be performed in specific build phases.
9. Build phase dependencies should be considered to ensure the correct execution order and avoid issues.
10. Documentation and communication should be provided to inform developers and team members about the purpose and configuration of each build phase.

****Alternative build servers**:**

1. Alternative build servers are other tools that provide similar functionality to Jenkins.
2. Examples of alternative build servers include Travis CI, CircleCI, TeamCity, Bamboo, and GitLab CI/CD.
3. These build servers offer various features and integrations, and their selection depends on project requirements and team preferences.

4. Alternative build servers may have different user interfaces, configuration approaches, and pricing models.
5. Migration from one build server to another may require adjustments to the build configurations and workflows.
6. Proper evaluation and comparison of alternative build servers are important to choose the most suitable one for the project.
7. Community support, documentation, and availability of plugins or integrations should be considered in the selection process.
8. Compatibility with existing infrastructure, version control systems, and development tools is crucial for a smooth transition.
9. Scalability, performance, and reliability of the alternative build server should be evaluated to handle the project's build load.
10. Training and onboarding may be required to familiarize the team with the chosen alternative build server.

****Collating quality measures**:**

1. Collating quality measures refers to collecting and analyzing various metrics related to the quality of the software product.
2. Quality measures can include code coverage, code complexity, test results, security vulnerabilities, and performance metrics.
3. Tools like SonarQube, Jenkins plugins, or specialized testing frameworks can be used to collect quality measures.
4. Quality measures provide insights into the overall health and stability of the software product.
5. Analysis of quality measures helps identify areas for improvement, potential issues, and code quality violations.
6. Trends and historical data of quality measures can be used to track progress and monitor the impact of quality improvement efforts.
7. Quality measures can be used to enforce coding standards, identify areas for refactoring, or prioritize bug fixes.
8. Integration of quality measures into the build process allows for continuous monitoring and feedback on code quality.
9. Dashboards, reports, and visualizations can be utilized to present quality measures in a clear and actionable manner.
10. Collaboration and communication among team members are essential to interpret and act upon the insights provided by quality measures.

DevOps UNIT-5

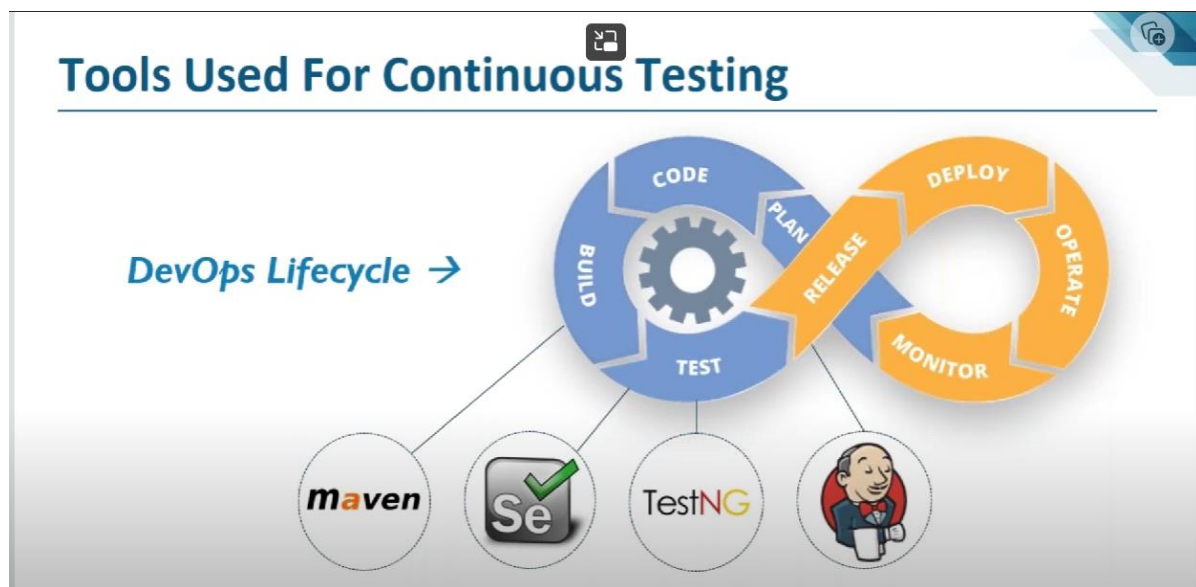
Software testing is the process of evaluating a software product to identify any errors, gaps, or defects. It is an important part of the software development process, and it can help to ensure that the software is fit for purpose.

Various types of testing

- **Unit testing:** Unit testing is the process of testing individual units of code. This type of testing is typically performed by developers, and it can help to identify errors in the code.
- **Integration testing:** Integration testing is the process of testing how different units of code interact with each other. This type of testing is typically performed by testers, and it can help to identify errors in the interaction between different units of code.
- **System testing:** System testing is the process of testing the entire software product. This type of testing is typically performed by testers, and it can help to identify errors in the overall functionality of the software product.
- **Acceptance testing:** Acceptance testing is the process of testing the software product to ensure that it meets the requirements of the customer. This type of testing is typically performed by the customer, and it can help to ensure that the software product is fit for purpose.
- **End-to-end testing:** End-to-end testing is the process of testing the entire software product from start to finish. This type of testing is typically performed by testers, and it can help to identify errors in the flow of the software product.
- **Performance testing:** Performance testing is the process of testing the performance of the software product. This type of testing is typically

performed by testers, and it can help to identify errors in the performance of the software product.

- Security testing: Security testing is the process of testing the security of the software product. This type of testing is typically performed by testers, and it can help to identify errors in the security of the software product.
- Usability testing: Usability testing is the process of testing the usability of the software product. This type of testing is typically performed by testers, and it can help to identify errors in the usability of the software product.



Different Testing Types In Continuous Testing

Functional Testing

Ensures that the software has the required functionality mentioned in its requirements.

Unit Testing

Smaller units/ components of a software application are tested independently for checking their functionality.

Load Testing

Process of testing the performance of the software application during heavy load/ usage.

Stress Testing

Process of determining the break point of the software application when under stress. (maximum load)

Integration Testing

Process where a group of components which are combined/ integrated to produce an output are tested.

Regression Testing

Testing the functionality of the entire application when even a small component has been modified.

Automation testing: is the use of software to execute tests on other software. It is a way to automate the manual testing process, which can save time and money. Automated testing can also improve the accuracy and efficiency of testing.

Manual testing: is the process of testing software manually by a human tester. It involves following a set of test cases and manually executing the steps in the test cases. Manual testing can be time-consuming and error-prone, but it can be more flexible than automated testing.

Here is a table that summarizes the key differences between automation testing and manual testing:

Automation testing	Manual testing
1. Uses software to execute tests	1. Human tester executes tests manually
2. Can save time and money	2. Can be more flexible
3. Less error-prone	3. More error-prone
4. Can be used to test more complex scenarios	4. Can be used to test more edge cases
5. Requires more upfront investment	5. Can be started more quickly

The best approach to testing depends on the specific needs of the project. For example, if the software is complex and time-sensitive, then automation testing may be a good option. However, if the software is less complex and there is more flexibility in the testing schedule, then manual testing may be a better option.

Ultimately, the decision of whether to use automation testing or manual testing should be made on a case-by-case basis, taking into account the specific needs of the project.

Here are some more points in the box for automation of testing pros and cons:

Pros	Cons
Reduces manual effort and saves time	Requires high initial cost and investment
Increases test coverage and accuracy	Requires maintenance and updates of test scripts
Improves test feedback and quality	Requires technical skills and expertise
Supports parallel and cross-browser testing	Has limitations and challenges of tools
Integrates with CI/CD and testing frameworks	May not replace human judgment and intuition
Enhances reusability and reliability of test scripts	May introduce new errors or bugs in the test code
Enables faster delivery and deployment of software	May not cover all scenarios and edge cases
Facilitates continuous testing and monitoring	May not handle complex and dynamic applications

Selenium - Introduction

Selenium is a suite of tools for automating web browser testing. It is a popular tool for testing web applications, and it is supported by a wide range of programming languages.

Selenium can be used to automate a wide range of tasks, including:

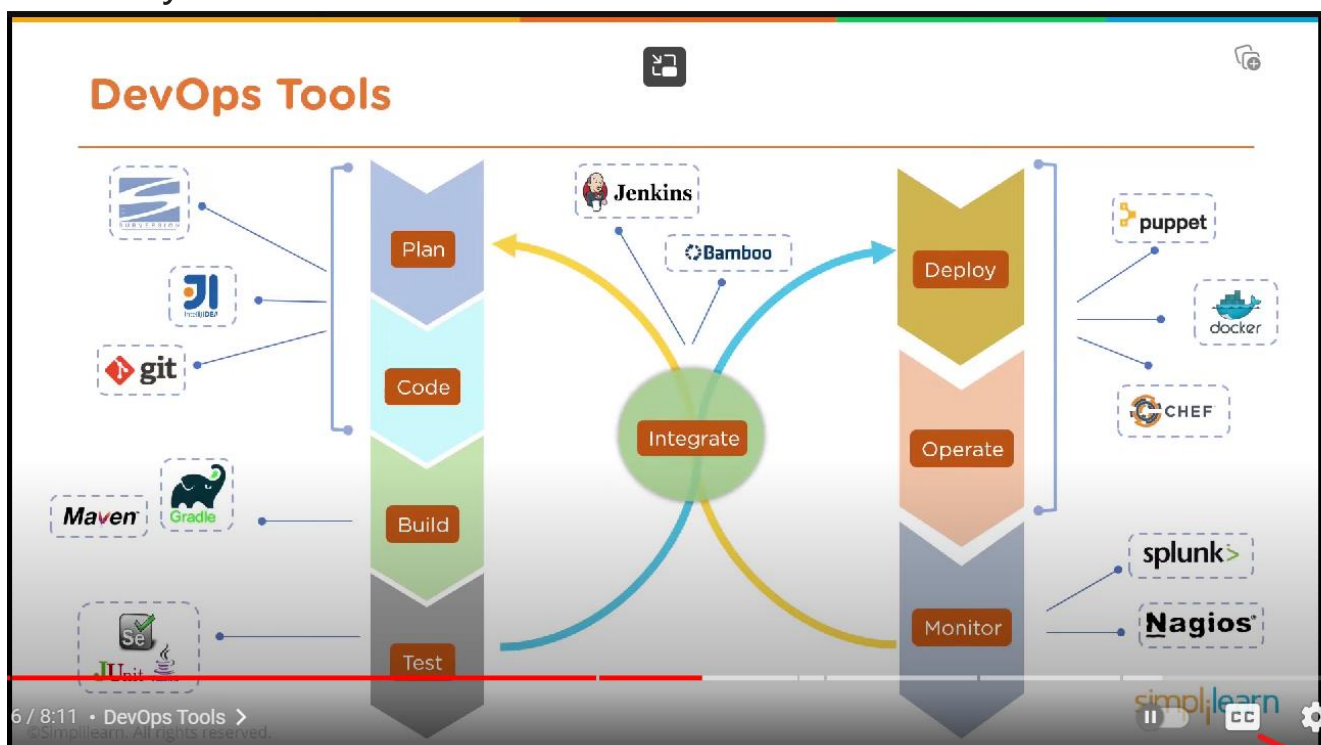
- Navigating to web pages: Selenium can be used to navigate to web pages, and it can also be used to interact with the elements on those pages.

- Submitting forms: Selenium can be used to submit forms, and it can also be used to verify the results of form submissions.
- Testing the behavior of web pages: Selenium can be used to test the behavior of web pages, and it can also be used to verify that web pages are behaving as expected.

Selenium features

Selenium has a number of features that make it a powerful tool for automating web browser testing, including:

- Cross-platform support: Selenium is supported by a wide range of operating systems and browsers.
- Open source: Selenium is an open source tool, which means that it is free to use and modify.
- Active community: Selenium has a large and active community, which means that there is a lot of support available for the tool.
- Documentation: Selenium has extensive documentation, which makes it easy to learn how to use the tool.



JavaScript testing

JavaScript is a scripting language that is used to add interactivity to web pages. It is a powerful language that can be used to create complex animations, user interfaces, and other interactive features.

JavaScript testing is the process of testing JavaScript code. This type of testing is typically performed by testers, and it can help to identify errors in the JavaScript code.

There are a number of different ways to test JavaScript code, including:

- **Manual testing:** Manual testing is the process of testing JavaScript code by hand. This type of testing can be time-consuming and error-prone.
- **Unit testing:** Unit testing is the process of testing individual units of JavaScript code. This type of testing is typically performed by developers, and it can help to identify errors in the JavaScript code.
- **Integration testing:** Integration testing is the process of testing how different units of JavaScript code interact with each other. This type of testing is typically performed by testers, and it can help to identify errors in the interaction between different units of JavaScript code.

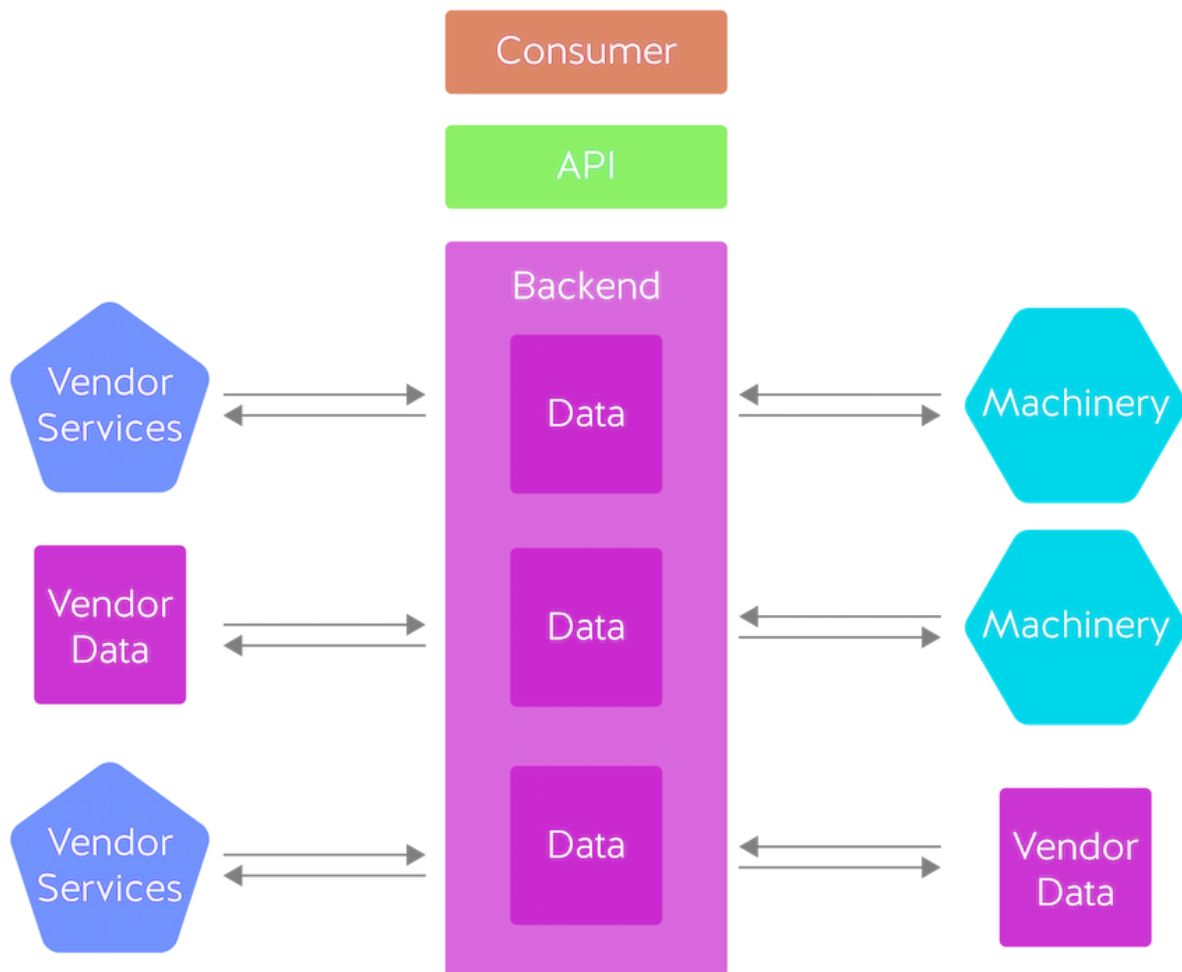
Testing backend integration points

Backend integration points are the interfaces between the backend and other systems, such as the database, the front-end, or other microservices. These points are important to test because they ensure that the backend is able to communicate with other systems and that the data is being transferred correctly.

There are a number of different ways to test backend integration points. One common approach is to use unit tests to test the individual components of the backend. Unit tests are typically written in isolation from other systems,

so they can be used to test the behavior of the backend without having to worry about the external dependencies.

Another approach to testing backend integration points is to use integration tests. Integration tests test the interactions between the backend and other systems. These tests can be more complex than unit tests, but they can provide a more comprehensive view of the behavior of the backend.

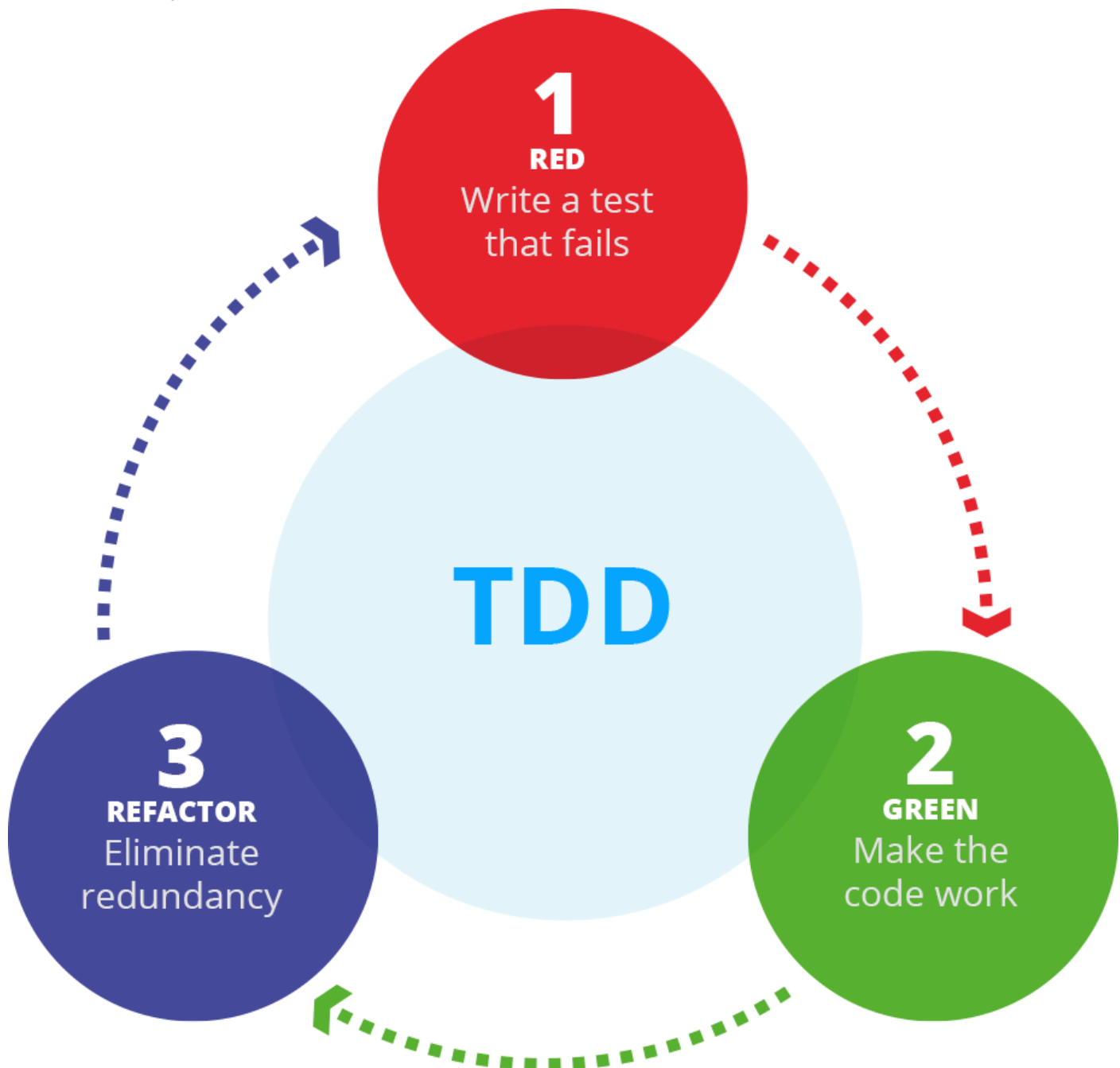


Test-driven development (TDD)

TDD is a software development process that involves writing tests before writing the code. The tests are used to define the desired behavior of the code, and the code is then written to pass the tests.

TDD has a number of benefits, including:

- Increased code quality: TDD forces the developer to think about the desired behavior of the code before writing it, which can lead to better code quality.
- Reduced defects: TDD can help to reduce defects in the code by catching errors early in the development process.
- Improved documentation: The tests can be used as documentation for the code, which can make it easier to understand and maintain.



REPL-driven development (RDD)

RDD is a development process that involves using a REPL (read-evaluate-print loop) to interactively develop code and tests. The REPL allows the developer to experiment with code and see the results immediately, which can help to improve the development process.

RDD has a number of benefits, including:

- Increased productivity: RDD can help to increase productivity by allowing the developer to iterate on code more quickly.
- Improved understanding: RDD can help the developer to better understand the code by allowing them to see the results of their changes immediately.
- Reduced defects: RDD can help to reduce defects in the code by catching errors early in the development process.

Comparison of TDD and RDD

TDD and RDD are both development processes that involve writing tests before writing the code. However, there are some key differences between the two approaches.

TDD is a more structured approach, with the tests being written before the code. RDD is a more flexible approach, with the tests being written alongside the code.

TDD is often seen as a more disciplined approach, while RDD is often seen as a more creative approach.

The best approach to use depends on the specific needs of the project and the preferences of the developer.

Here is a table that summarizes the key differences between TDD and RDD:

TDD

Tests are written before the code.

More structured approach.

Often seen as a more disciplined approach.

RDD

Tests are written alongside the code.

More flexible approach.

Often seen as a more creative approach.

Conclusion

Testing backend integration points, test-driven development, and REPL-driven development are all important techniques for ensuring the quality of software. The best approach to use depends on the specific needs of the project and the preferences of the developer.

Deployment of the system

Deployment is the process of installing and configuring software on a target environment. It is an important part of the software development lifecycle, and it can be a complex and time-consuming process.

The deployment process typically involves the following steps:

1. **Planning:** The first step is to plan the deployment. This involves identifying the target environment, the software to be deployed, and the deployment methods to be used.
2. **Acquisition:** The next step is to acquire the software to be deployed. This may involve purchasing the software, downloading it from a public repository, or building it from source code.
3. **Configuration:** The software must then be configured for the target environment. This may involve setting environment variables, installing dependencies, or creating configuration files.

4. **Deployment:** The software is then deployed to the target environment. This may involve copying the software files to the target environment, installing the software using a package manager, or using a deployment tool.
5. **Monitoring:** The deployment is then monitored to ensure that it was successful. This may involve checking the status of the software, verifying that the software is functioning correctly, and collecting performance metrics.

Deployment systems

A deployment system is a software application that automates the deployment process. Deployment systems can help to simplify the deployment process, and they can also help to ensure that the software is deployed consistently across all target environments.

There are a number of different deployment systems available, each with its own strengths and weaknesses. Some popular deployment systems include:

- **Chef:** Chef is a configuration management tool that can be used to automate the deployment of software. Chef is a popular choice for large-scale deployments, and it is known for its flexibility and scalability.
- **Puppet:** Puppet is another configuration management tool that can be used to automate the deployment of software. Puppet is a popular choice for small- to medium-sized deployments, and it is known for its ease of use and simplicity.
- **Ansible:** Ansible is a configuration management tool that uses a push-based model for deployment. Ansible is a popular choice for deployments that involve a large number of servers, and it is known for its speed and efficiency.
- **Salt Stack:** Salt Stack is a configuration management tool that uses a master-agent model for deployment. Salt Stack is a popular choice for deployments that require a high degree of scalability and reliability.

- Docker: Docker is a containerization tool that can be used to deploy software. Docker containers are lightweight and portable, and they can be used to deploy software on a variety of platforms.

The best deployment system to use depends on the specific needs of the project. Some factors to consider when choosing a deployment system include the size and complexity of the deployment, the number of target environments, and the budget.

Virtualization stacks

A virtualization stack is a collection of software that allows for the creation of virtual machines. Virtual machines are software emulations of physical computers, and they can be used to run different operating systems and applications on the same physical hardware.

Virtualization stacks can be used to simplify the deployment process, and they can also help to improve the security and scalability of deployments.

Some popular virtualization stacks include:

- VirtualBox: VirtualBox is a free and open-source virtualization software. VirtualBox is a popular choice for personal use, and it is also used by some businesses.
- VMware: VMware is a commercial virtualization software. VMware is a popular choice for businesses, and it is known for its scalability and performance.
- Hyper-V: Hyper-V is a virtualization software that is included in Windows Server. Hyper-V is a popular choice for businesses that use Windows Server, and it is known for its ease of use and integration with other Microsoft products.

Code execution at the client

Code execution at the client is a deployment method in which the software is deployed to the client's computer and then executed there. This method is often used for software that requires a lot of user interaction, such as web applications.

Code execution at the client can be a good choice for deployments that require a high degree of flexibility, as the software can be customized to the specific needs of the client. However, code execution at the client can also be more complex and time-consuming than other deployment methods.

Puppet master and agents

Puppet is a configuration management tool that uses a master-agent model for deployment. The Puppet master is a central server that manages the configuration of the agents. The agents are the computers that are being configured.

The Puppet master and agents communicate with each other using the Puppet protocol. The Puppet protocol is a simple text-based protocol that is easy to understand and use.

Ansible

Ansible is a configuration management tool that uses a push-based model for deployment. Ansible is a lightweight tool that is easy to learn and use.

Ansible uses SSH to communicate with the target servers. SSH is a secure protocol that is used to access remote computers.

Ansible is a popular choice for deployments that involve a large number of servers, as it can be used to deploy software quickly and efficiently. Ansible is also a good choice for deployments that require a high degree of security, as SSH is a secure protocol.

Deployment tools: Chef, Salt Stack and Docker

Chef, Salt Stack and Docker are all deployment tools that can be used to automate the deployment of software. Chef and Salt Stack are both configuration management tools, while Docker is a containerization tool.

Configuration management tools are used to manage the configuration of servers. Containerization tools are used to create and manage containers.

Chef is a popular choice for large-scale deployments, as it is known for its flexibility and scalability. Salt Stack is a popular choice for deployments that require a high degree of scalability and reliability. Docker is a popular choice for deployments that involve a large number of containers, as it can be used to deploy software on a variety of platforms.

The best deployment tool to use depends on the specific needs of the project. Some factors to consider when choosing a deployment tool include the size and complexity of the deployment, the number of target environments, and the budget.